ABSTRACT

Cyber threats are becoming increasingly sophisticated at an exponential rate, and no organization can defend itself in isolation against zero-day attacks. A strong collective defense can be built if network telemetry is shared across organizations, but strict data privacy regulations (e.g. GDPR), and the high risk of leaking Personally Identifiable Information (PII), make central pooling of raw network traffic impossible. Existing collaborative intrusion detection architectures either violate sensitive data privacy by centralization, or impose strict isolation, thus making it impossible to find attack patterns across organizations.

To address a fundamental challenge, we present ACTIS, the Agentic Collaborative Threat Intelligence System. ACTIS applies Trust-Aware Federated Learning and Agentic Retrieval-Augmented Generation in a privacy-preserving fashion over the edge to the cloud. It enables us to train the intrusion detection models across different enterprise and IoT environments while keeping the raw data confidential. To deal with non-identical (non-IID) data with significant variability, we employ a FedProx-based training scheme with a Trust-Aware FedAvg algorithm that penalizes underperforming or compromised nodes when needed.

In addition, ACTIS features an LLM-powered agentic privacy engine. It recognizes network issues with MITRE ATT&CK tactics, ensuring no personal data is leaked due to rigorous regex filtering. The data is then cleaned, and threat intelligence is converted into secure embeddings stored in a centralized vector database. ACTIS also achieved high scores with 98.82% federated accuracy, 92.52% zero-day detection, and a perfect 0.0% PII leakage. Its BERT Score F1 was 0.8710, beating other systems such as ReGAIN, Tri-LLM, and CyberRAG. The Immunity Engine uses a technique called Maximal Marginal Relevance search to automatically propagate new defenses across the network, turning unknown threats into rules for firewalls.

Chapter 1

INTRODUCTION TO AGENTIC COLLABORATIVE THREAT INTELLIGENCE SYSTEM

Cyber threats are getting more sophisticated, and enterprise and IoT networks are inundated with new zero-day attacks. Traditionally, enterprises have relied on standalone defenses, such as localized Intrusion Detection Systems (IDS). However, these solo systems are always behind modern threat actors that now use coordinated, cross-network attack strategies. Sharing network telemetry and attack signatures could greatly enhance collective defense efforts, but here is the catch: strict data privacy rules, such as GDPR and CCPA, make it impossible to share raw network data due to fears of leaking Personally Identifiable Information (PII).

1.1 Introduction

The explosion of connected environments from large enterprise rollouts to cloud services and the ever-growing IoT has significantly expanded the attack surface in today's digital world. The cybersecurity landscape has shifted from simple, single malware to highly sophisticated, coordinated, and automated attacks. These bad actors leverage things like advanced persistent threats (APTs), polymorphic malware, and automated hacking tools to circumvent traditional safety nets. And perhaps the biggest concern, zero-day attacks, which hit before anyone even has a chance to figure out what they are or how to remediate them.

Companies like to stand alone in their defenses, so if one company gets hit by a zero-day attack, it can be hit again and again. The people defending us just aren't talking to each other; the news of attacks is often too late, after it's already caused problems and someone has had to manually write a report. To even the odds, we need to share threat intelligence a lot more. Connecting up all that info from different nets could give us one powerful shield against hacks. Spot an attack in Node A, defend Nodes B and C immediately. There are massive advantages to a team defense approach, but it’s hard to do. There are big privacy concerns in the way, as well as the issue of data centralization. Regulations such as Europe’s GDPR, California’s CCPA, and HIPAA place strict rules on how data is handled and stored, making it very hard to move network information around. Network traffic, even raw packet headers and NetFlow stats, often contains embedded Personally Identifiable Information (PII), proprietary corporate data, intellectual property, and sensitive user credentials. So, organizations are legally and ethically barred from combining their raw network logs into a centralized, cloud-based repository for collaborative machine learning analysis. Existing collaborative architectures impose an unfortunate trade-off: either organizations must centralize their data, violate privacy, and create a large, lucrative target for data breaches, or they must isolate data, crippling the ability of ML models to detect attack patterns across organizations.

1.2 Overview of Agentic Collaborative Threat Intelligence

Until now, collaborative threat intelligence has relied on centralized architectures such as Data Lakes or Security Information and Event Management (SIEM) aggregators. This means that organizations that participate in threat intelligence need to upload raw network logs, packet captures (PCAP) and telemetry data to a central authority for analysis. However, this creates a single point of failure and a huge target for cybercriminals. More importantly, collecting raw data centrally is a serious breach of modern privacy compliance requirements such as GDPR, CCPA and HIPAA because network logs often contain embedded Personally Identifiable Information (PII) and proprietary operational data. ACTIS replaces this outdated paradigm with a decentralized, privacy-first architecture that

uses autonomous AI workflows. ACTIS operates directly at the edge of the network, ensuring that raw data stays with the host organization, while building its defense on four advanced technical pillars.

Federated Learning (FL) for Decentralized Intelligence

ACTIS addresses data localization through Federated Learning. Instead of transmitting sensitive network traffic to a central server for model training, we deliver the central machine learning model directly to the local data. For example, network nodes such as enterprise servers and IoT devices train their own local Intrusion Detection System (IDS) models using their live traffic. After a local training cycle is completed, they only send the mathematical aggregation of anonymized weight updates to the central server. Consequently, the global IDS model can learn a variety of attack patterns globally, while the actual network packets remain securely within each organization's perimeter.

Agentic Privacy Engines for Autonomous Sanitization

ACTIS then uses Agentic AI to perform contextual threat analysis and sanitize data. If the local IDS detects something suspicious, ACTIS sends Large Language Models (LLMs) as autonomous, goal-oriented agents to deal with the situation. These Agentic Privacy Engines take care of the whole process on their own. The first one is a "Analyzer Agent" that looks at the context and payload of the problem and maps the activity to frameworks such as MITRE ATT&CK.

Differential Privacy (DP) for Mathematical Guarantees

Differential Privacy, or DP, is the next line of defense to harden defenses against smart machine learning attacks. Even if the personal info is stripped out and you're just sharing model weights or summaries, sneaky attackers might use tricks like membership inference or model inversion to guess at the original data. ACTIS counters this by adding some calculated noise, specifically Gaussian noise, to what's sent out the threat outlines and updates. This gives formal (ϵ, δ)-differential privacy protections, which basically offer a math-

backed safeguard that makes it super hard for someone in the middle to figure out if a specific person's or company's data was part of the mix.

Digital Immunity via Agentic RAG

The final piece of this security puzzle is digital immunity in an agentic retrieval-augmented generation setup. Once DP has sanitized and secured the info, it is translated into clean vector formats and stored in a single repository called ChromaDB. When a node encounters new, potentially harmful info, the RAG system kicks in. It compares this against the large database, finds the matching segments, and generates customized defenses that can be rapidly deployed across the network. So, the defense is both broad and highly flexible and fast.

1.3 Literature Survey

The architecture of the Agentic Collaborative Threat Intelligence System (ACTIS) is based on recent advances in distributed machine learning, privacy-preserving techniques, and generative AI. To understand the background and the gaps that ACTIS aims to address, the literature can be grouped into four major categories: Federated Learning for network security, privacy protection, large language models in cyber defense, and RAG for automated defenses. The most influential of these is Federated Learning (from [1]), as it supports ACTIS's approach to handling client penalties and semantic drift at the nodes. RAG is critical to converting threat intel into practical defense rules, with ReGAIN [2] demonstrating how RAG can be used for interpretable network traffic analysis, and architectures like CyberRAG [3] enabling agent-based architectures with classification tools and extensive reporting. These major contributions have greatly influenced the design of the Immunity Engine in ACTIS.

The world of federated network security faces a big challenge in statistical heterogeneity, especially with non-IID data distributions. To address this, recent applications [4] used federated learning for highly imbalanced data, such as in

IoT contexts, which highlights the importance of robust local processing before data aggregation. Reviews [5] evaluated the capabilities of large language models in cybersecurity and think we should leverage them for structured threat information extraction. Research [6] on multi-agent systems showed how to identify active threats and adapt defenses, which proved highly beneficial in informing ACTIS's Agentic Orchestration engine. One notable feature of ACTIS is its ability to prevent data leakage during automatic reporting.

Existing research [7] has demonstrated that plain LLMs can develop vulnerabilities for data theft and other risky memorizations, requiring secure backups. Similarly, context-aware transformers [8] are capable of detecting unusual data never before seen by comparing it to huge pre-trained sets, aligning perfectly with the zero-day classifiers of ACTIS to improve alarm triggering. To mitigate leakage risks, research on automated redaction techniques [9] reduces leaks to zero while maintaining the needed context. ACTIS builds upon these learnings by using a closed retry-loop with strict validator protocols and LLM generation to develop their Agentic Privacy Engine. For improving vulnerability analysis, frameworks such as RAG-Sec [10] incorporate real-time intelligence feeds to fortify solo analyses, akin to how ACTIS searches CVE databases and MITRE ATT&CK tactics.

Classic Intrusion Detection Systems tend to rely on a centralized architecture, which poses huge privacy problems. Previous works [12, 25] have shown how to secure federated IDS architectures. It has been shown that distributed anomaly detection, using federated learning, is able to achieve the same performance as centralized systems without compromising accuracy. Recent works [13, 14] showed that lightweight multi-layer perceptron neural networks deployed at the network edge work very well. They also proved that federated learning is able to scale across IoT devices with constraints. Surveys [15] detail how to solve data mismatches between federated learning participants. Also, ACTIS solves problems of hacked nodes by leveraging ideas [16]. Such scores indicate the use of dynamic reliability scores based on historical performance data. They apply

this approach in their unique Trust-Aware FedAvg aggregation method. Targeted research [17] demonstrated the injection of differential privacy noise directly into semantic vector spaces . Additionally, studies [18] proved this by demonstrating the effectiveness of diff priv in defending against attacks on dense vectors, such as those in ACTIS's DP layer.

Another paper examined Local Differential Privacy [19]. It compared local and central DP approaches in the context of active federated training. This paves the way for applying DP to local gradients before they reach the server at all. Also, the use of generative AI for automatic penetration testing [20] really cements the notion of bringing theory into practice for security. Linking logic AI to real world infra also strengthens this transition [21].

And finally, new research [23] found that searching for similarities in vector databases could identify the same attacks traveling across networks. This powers ACTIS's Immunity Engine. Here, ChromaDB finds matches to build firewall rules to turn smart data into real protection, spreading immunity throughout orgs.

Reviews of Federated Learning for the Industrial Internet of Things described the architectural requirements for fast updates in SCADA and IIoT networks. Smart configurations with deep architectures such as MLPs and CNNs performed better when integrated into federated loops, outperforming stand-alone local models. This validates the broad structure ACTIS uses at the network edge. Additional studies identified threats such as Sybil attacks and proposed methods to limit trust boundaries and resist poisoned data through resilient aggregation techniques. Reviews considering the overall landscape examined threats such as model inversions and poisoning attacks, which constitute the primary threat model for decentralized systems. The DeepFed project demonstrated how all this works in practice, providing a baseline for connecting large telemetric data from physical networks with deep learning frameworks.

Later studies [34] analyzed the trade-offs between utility and privacy for differentially private federated systems. Another study [35] proposed a

combination of Multi-Party Computation (MPC) with differential privacy to increase security in untrusted environments. They demonstrated how this can work in theory within the ACTIS network. Then, there was the FedProx algorithm [36], which took care of the data variations across non-IID participants. It introduced a proximal regularization term in the loss function, ensuring that the model would still converge even with the huge differences between enterprise and IoT dataset distributions in ACTIS. Finally, key papers [38] proved that the addition of parametric noise in federated networks could limit the amount of data that can be inverted, thanks to differential privacy. This gives the math backing ACTIS needs for its privacy assurances on shared model updates and embedded threats.

To future-proof the ACTIS network, advanced privacy mechanisms beyond standard Differential Privacy are used as the theoretical backbone. The authors also investigated Local Differential Privacy [19], comparing local privacy bounds in active federated training with central DP methods. They concluded that DP guarantees may be directly applied to the individual gradient data before leaving the local node – something that ACTIS could easily achieve with its modular DP layer design. Moreover, hybrid solutions for privacy-preserving federated learning [35] advocate for a combination of Multi-Party Computation (MPC) and DP to guarantee security even among untrusted participants.

1.4 Motivation

The main motivation for ACTIS stems from a fundamental problem of modern cybersecurity - there is a strong contradiction between the pressing need for everyone to protect each other on a global scale and strict rules on data privacy. As new cyber threats emerge, security systems require a variety of network data to detect stealthy attacks. However, laws such as GDPR and CCPA prohibit sharing information containing personal data or company secrets. As a consequence, organizations find themselves in a difficult position - they cannot legally merge the essential data to develop improved Intrusion Detection Systems. They may, as a result, miss out on attacks occurring beyond their networks.

The lack of secure data sharing has created a big problem for those trying to defend against cyber-attacks. Currently, organizations are fighting against highly connected and cooperative attackers with isolated defense systems. Cybercriminals nowadays work together in complex networks, rapidly spreading new attacks and sharing secrets on hidden forums. Organizations struggle to protect themselves because they’re restricted from easily sharing crucial threat info due to worries about data leaks, legal issues, and harm to reputation.

Also the sharing of threat intelligence can be very slow and inefficient. When a company detects a new threat, such as a zero-day attack, it has to go through a lot of steps. It has to analyze the breach very thoroughly, identify all indicators of compromise, prevent any personal information from being leaked,

and then prepare a comprehensive report.

This can take days or even weeks. But in the world of modern fast-spreading dangers, that's way too slow. Criminals get to zoom in, find fresh victims, and strike again before anyone else can fully protect themselves against the attack. So, we really need an automatic system that works at machine speed to get past those manual holdups. The cybersecurity community needs something that can spot new threats right away, figure out what makes them tick on its own, and quickly set up defenses elsewhere – all while keeping the data totally private. That's where ACTIS comes in. It wants to connect this gap using Trust-Aware Federated Learning, AI-driven data cleaning, and Differential Privacy. ACTIS aims to show that strong, auto cross-organizational threat protection is actually possible while maintaining privacy. This would move the world from isolated and reactive to a more connected and proactive defense stance.

1.5 Problem Statement

We need to design a privacy-preserving, collaborative Intrusion Detection System that allows multiple, heterogeneous network nodes (Enterprise and IoT) to collaboratively train a strong IDS model and automatically share zero-day threat intel, while dealing with non-IID data distribution, penalizing unreliable nodes, and ensuring that no PII leaks during sharing. It should also be capable of creating and distributing deployable perimeter defense setups by itself.

1.6 Objective

- Decentralized IDS Training: To implement a PyTorch-based IDS using the FedProx algorithm to effectively train across statistically heterogeneous (non-IID) enterprise and IoT datasets.
- Resilient Aggregation: To develop a "Trust-Aware FedAvg" strategy at the central server to dynamically penalize underperforming or compromised nodes using empirical loss metrics.
- Agentic Privacy & PII Sanitization: To employ an LLM-driven Agentic Privacy Engine (using LangChain and Gemini) with Regex fail-safes to guarantee 0% PII leakage while mapping threats to MITRE ATT&CK

tactics.

- Mathematical Privacy Guarantees: To engineer a Differential Privacy (DP) layer that applies calibrated Gaussian noise to semantic embeddings, ensuring formal ((\epsilon, \delta))-DP guarantees.
- Automated Zero-Day Immunity: To build a Global Federated RAG Knowledge Base (ChromaDB) that powers an Immunity Engine capable of autonomously generating deployable perimeter defenses (e.g., firewall rules).

1.7 Scope

This project will work with common datasets such as NF-CSE-CIC-IDS2018-v2 for enterprise environments and NF-BoT-IoT-v2 for IoT environments to collect NetFlow v2 data. It will include the development of a PyTorch-based MLP to identify local threats, as well as a custom federated aggregation server. We will also use Google's Gemini 2.0 API through LangChain to analyze threats and clean PII, then transform the cleaned data into vectors using SentenceTransformers and store them in ChromaDB. The team will also generate rules to automatically defend endpoints, like iptables and ModSecurity configurations.

1.8 Methodology

ACTIS is an end-to-end edge-to-cloud security pipeline for effective intrusion detection and privacy preservation. It consists of six interconnected phases from the network edge to the central server.

Heterogeneous Data Processing

ACTIS starts by collecting data at the network edge, where enterprise and IoT devices continuously generate live NetFlow traffic. This raw data can differ significantly based on hardware and configurations, so it undergoes a rigorous normalization process to generate a uniform 41-dimensional feature vector. Additionally, different organizations may use different names for similar attacks. ACTIS translates all local attack labels into a universal taxonomy, enabling it to consistently describe attack types such as DDoS, traditional DoS, and botnet

activity. Only after this standardization does ACTIS employ machine learning.

Local Model Training (FedProx)

Once data is normalized, each node trains a deep Multilayer Perceptron (MLP) architecture on its local dataset. A major challenge in federated settings is that local data is not IID (Independent and Identically Distributed). For instance, an IoT camera will not observe the same type of traffic as an enterprise web server. To prevent local models from diverging too much or not detecting global attack patterns, ACTIS uses the FedProx algorithm. This algorithm modifies the traditional local loss function by adding a mathematical proximal term.

Trust-Aware Global Aggregation

Following local training, the nodes send their updated model weights—none of which contain raw network data—to the central FL server, where the server does not simply average these weights together because that might allow harmful data to slip through, but instead performs a Trust-Aware aggregation protocol. The FL server calculates a dynamic trust score for each node based on the generalization of its training loss when tested, combined with the volume of the node's sample data, to produce the final global model, effectively cutting out any nodes that are acting weird, performing poorly, or being compromised maliciously.

Agentic Privacy Engine

In case a local IDS identifies a certain fishy anomaly that could be a new attack, the system switches from prediction to deep dive analysis. The LLM-driven Agent Analyzer proceeds to analyze the metadata, payload features, and port activities of the anomalous data packets. After gathering all this data, it creates a standard threat report with respect to the MITRE ATT&CK techniques.

Differential Privacy Layer

Then an Agent Sanitizer comes into play to make sure all personal data is removed. It runs a deep cleaning pass on any IPs, MAC addresses, or inside-domain tags with complex regex, leaving only useful threat details. For extra protection, the cleaned report is processed by a Differential Privacy layer. This part converts the report into a dense, multi-dimensional vector and adds finely tuned Gaussian noise. This noise prevents attackers from reversing the anonymization even if they intercept the shared intel.

1.9 Organization of the Report

- Chapter 2 — Theory and Concepts of ACTIS provides the theoretical background covering Federated Learning, deep learning architectures, LLMs, RAG, and Differential Privacy.
- Chapter 3 — Software Requirement Specification of ACTIS details the complete functional, performance, software, and hardware requirements governing the development and deployment of the framework. It also defines the design constraints imposed by the privacy-by-design principles central to ACTIS.
- Chapter 4 — High-Level Design Specification of ACTIS explains the three-tier system architecture and the architectural strategies including hyperparameter tuning, feature engineering, scalability, and evaluation. It presents a hierarchical Data Flow Diagram analysis at Level 0, Level 1, and Level 2 for each core processing module.
- Chapter 5 — Detailed Design of ACTIS presents the module-level decomposition of the framework through a complete structure chart covering all 22 system modules. It provides detailed design descriptions, parameter tables, and edge case handling for each component from data ingestion through federated aggregation.
- Chapter 6 — Implementation of ACTIS discusses the technical execution of the project, covering the programming language selection and all key libraries.
- Chapter 7 – Software Testing of ACTIS lays out the unit testing for the preprocessing, feature engineering, PII sanitization, and classifier modules using structured test cases.
- Chapter 8 – Experimental Results and Analysis then looks at ACTIS's performance across four federated nodes. This includes classification results, confusion matrices, ROC curves, privacy benchmarks, and zero-day detection rates. Also, it compares ACTIS to ReGAIN, Tri-LLM, and CyberRAG.
- Chapter 9 wraps things up by summarizing the project's major

contributions and laying out those final experimental results in detail.

Chapter 2

THEORY AND CONCEPTS OF AGENTIC COLLABORATIVE THREAT INTELLIGENCE SYSTEM

In this chapter we present the main theories and concepts of collaborative intrusion detection based on federated learning and deep learning approaches. We describe the current cyber threats, their effect on the decentralized networks, and the problem of automatic detection of zero-day attacks while ensuring data privacy. The chapter also describes how federated learning techniques such as FedProx and Agentic AI using Large Language Models (LLMs) have transformed threat intelligence. These innovations allow for strong and data-privacy-preserving analyses of network anomalies and attack patterns. It also explains the shift from classical centralized classification to decentralized deep learning for network monitoring. The remainder of today's talk is about challenges that need to be tackled, such as inconsistent statistical data, the threat of malicious nodes that try to sabotage things, and the absolute necessity of not losing any data while reporting threats. All of this creates the environment for the construction of secure, expandable, and automatic systems to protect against digital threats.

2.1 Introduction & Intrusion Detection systems

The contemporary method for protecting networks is using Intrusion Detection Systems (IDS). IDS tries to detect strange traffic or known threats. IDS generally exists in two flavors: signature based (for known malicious patterns) and anomaly-based.

2.2 Federated Learning in Cybersecurity

Federated Learning (FL) is a decentralized machine learning technique where the participating entities collaboratively learn a model without revealing their local data. This allows for the training of an Intrusion Detection System (IDS) over networks such as enterprise and IoT networks without exposing actual packet information. The conventional approach to FL is federated averaging (FedAvg) where a simple model is sent to all nodes, trained on local data, and the trained model weights are uploaded back to a centralized server to generate an improved global model that is sent back to all nodes. So, they all gain without revealing any personal data. Network traffic is very different, an enterprise's data flow is very different from IoT networks, right? This is called non-IID data. The standard FedAvg method is not very good with this. So, ACTIS uses FedProx, which adds a special term to the local loss function to keep things under control. This helps to prevent local models from straying too far from the main group, making everything work smoother in varied network settings. In addition to that, ACTIS has a Trust-Aware aggregation system. It figures out a trust score for each node using the node's error rate. If some nodes seem shady or are trying to mess with the system, this mechanism catches them.

To build strong ML models, we need truckloads of data. Usually, gathering all that info in one place – a data lake – does the trick. But it comes with big risks like privacy leaks, legal issues due to regulations like GDPR, and there's always that one weak spot that can crash the whole thing. ACTIS sidesteps all those hurdles by avoiding the central data lake idea. Instead, it adopts a more collaborative approach where multiple nodes work together but share the load and the risk. Each node trains the model locally on its data for several epochs but only sends mathematical weight updates to a central server. ACTIS prevents local models from diverging or forgetting attack patterns by using the FedProx algorithm. This adjusts the local loss function with a proximal term. Ultimately, it converts this info into a standardized threat report directly linked to MITRE ATT&CK tactics and techniques.

2.3 Deep Learning and Agentic AI Techniques in ACTIS

To achieve high accuracy in zero-day threat detection and make generating threat intel automated, ACTIS uses a combo of Deep Learning and Generative AI tech.

2.3.1 Multilayer Perceptron (MLPs) for Network Telemetry

For the critical prediction algorithm at individual local locations (both Enterprise and IoT), ACTIS uses PyTorch-based Multilayer Perceptrons (MLPs) which are a class of feedforward neural network with layers upon layers. Raw NetFlow data are fed continuously and then scaled rigorously (standards like Z-score or Min-Max are used to stop gradients from drowning each other out) leading to normalization into a big 41-dimensional feature vector.

The vector then moves into the deep hidden layers of the MLP, which usually contain 128, 64 and 32 neurons respectively. These layers use non-linear activation functions such as the Rectified Linear Unit (ReLU) to learn the complex, non-linear statistical properties of network traffic headers, for example, packet arrival times, byte distributions and flag frequencies. To prevent overfitting and maintain high accuracy, Dropout layers are used during the training of the MLP. This allows the MLP to accurately classify different types of traffic, ranging from benign data to DDoS floods, botnet activity and brute-force attacks. The Categorical Cross-Entropy is used to quantify the deviation of the predicted values from the actual ones. The optimizer (usually Adam) performs backpropagation to update the weight matrices. Importantly, in this federated setting, the regular loss function is modified using the FedProx algorithm. The modification introduces a proximal regularization term to prevent local models from drifting too far away from the global model maintained by the central server. This promotes consistency between various components of the distributed network.

The network is sprinkled with dropout layers. The last hidden layer is connected to an output layer that matches the dimensions of the threat taxonomy. The network uses a Softmax activation function to convert its numbers to a probability distribution for each attack class. If the probability for a malicious class exceeds the confidence threshold, the MLP triggers an anomaly alert to kick start the Agentic Privacy Engine. Figure 2.1 shows the MLP architecture.

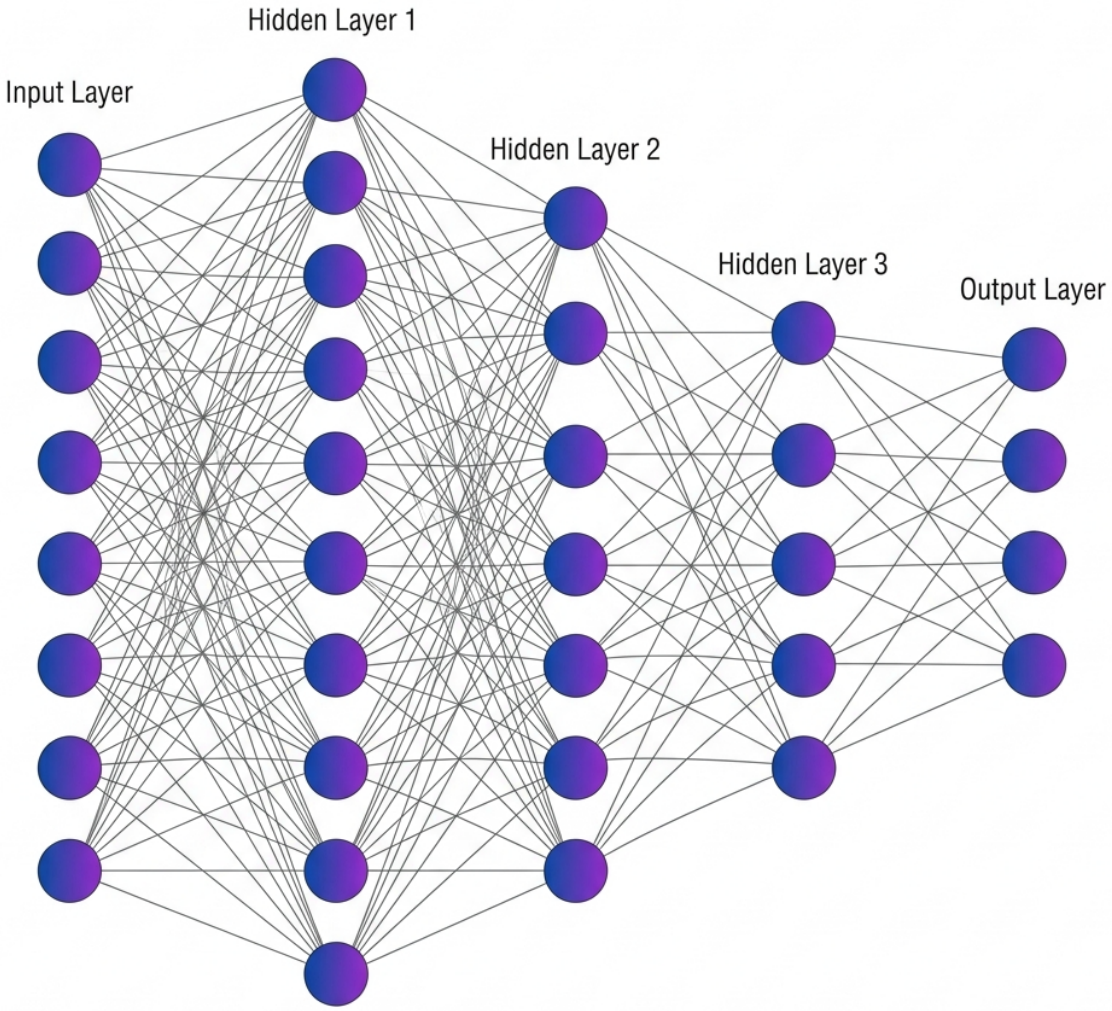

Fig 2.1 MLP Architecture

2.3.2 Large Language Models (LLMs) and Agentic Sanitization

MLPs are great for quick, numerical classifications based on stats, but they don't get the bigger picture. They can't create human-readable threat reports, nor do they understand the big strategy behind attacks. To fill this huge gap, ACTIS uses Large Language Models like Google's Gemini 2.0, run by the LangChain framework. The LLM here isn't just a chatbot it's a dynamic, goal-directed agentic privacy engine. If the local MLP IDS spots something fishy, an analyzer agent kicks in. This guy then goes to the LLM for help in checking out structural metadata, payload sizes, and port-targeting behaviors. After getting this info, it translates those details into standardized TTPs that line up perfectly with the MITRE ATT&CK framework.

However before the information is sent to the global network, a second layer is added. It is not safe to use LLMs to remove personal information because they can fail. This is why ACTIS uses a Sanitizer Agent with strict Regex validators – they are like clear rules, not guesses. It results in a "closed retry-loop": the LLM tries to send a report, but if it still has IPs, MAC addresses or internal domain names, the Regex engine stops it. Then the LLM has to try again until the report passes the test. This combination ensures that no private information leaks out and only what is important remains. The whole process begins when an MLP IDS detects something suspicious. Instead of discarding the raw data, the system triggers an Analyzer Agent. This tool analyzes the structure of the data, the size of payloads, the flag distributions and the targeted ports. Using the Gemini 2.0 model, the Analyzer translates all that information into a neat, strategical analysis that is easy to understand.

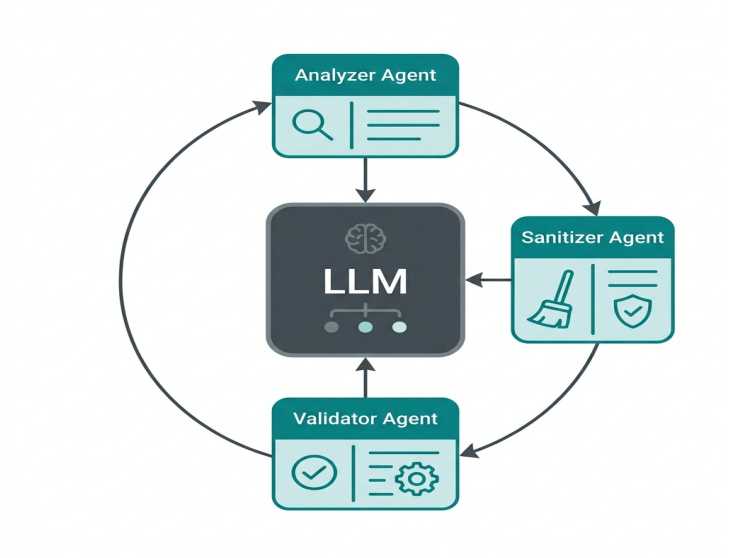

Fig 2.2 LLM Agentic System

2.3.3 Retrieval-Augmented Generation (RAG) and Differential Privacy

To make the clean, human-readable threat reports from the Agentic Privacy Engine machine-searchable, ACTIS uses advanced semantic vectorization techniques combined with Differential Privacy (DP). The sanitized text is initially processed through a Sentence Transformer model that encodes the text into a dense semantic vector space of 384 dimensions, where similar threat behaviors tend to form clusters. However, releasing even sanitized versions can still expose a potential attack surface for reverse engineering the original text with sophisticated adversarial techniques such as membership inference or model inversion attacks. To address this, ACTIS further processes the embeddings through a Differential Privacy layer. This step adds Gaussian noise to only the numerical parts of the vector. The scale of the noise is determined by a strict privacy budget, giving the attacker no sense of whether a given org's data was included in the intel, while still maintaining the larger context required for threat matching. These protected threat vectors are then sent to the global Federated Knowledge Base operated by a central ChromaDB instance, as shown in Fig 2.3.

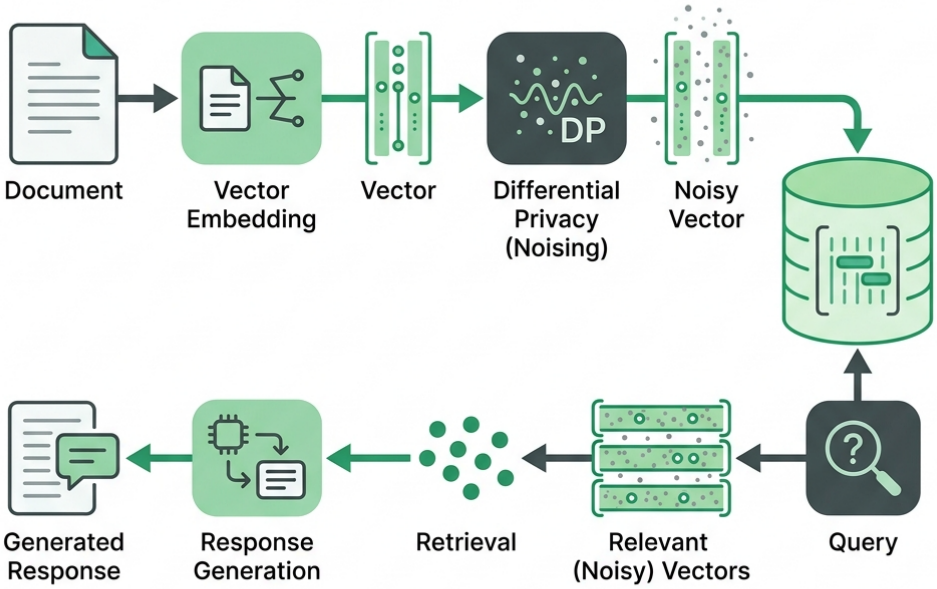

Fig 2.3 Rag Differential Privacy

2.3.4 Comparison and Applicability

Existing Deep Learning approaches are extremely effective but not suitable for multi-organizational applications due to privacy concerns. Conversely, stand-alone local ML models preserve privacy but are ineffective in detecting new unseen attacks. Our ACTIS system leverages FedProx for robust decentralized training, Local Model Logic for data sanitization, and RAG for defense, thus fulfilling all requirements. Combining these approaches, ACTIS addresses the challenges of strict data privacy and the immediate need for collaboration to combat unknown threats. This approach also encompasses traditional centralized data processing methods, the evolution of decentralized deep learning in network domains, and the advantages of RAG. It also highlights important challenges such as heterogeneous data, complex malicious actors, and privacy-preserving threat sharing.

2.4 Machine Learning Algorithms for Classification

Machine learning algorithms are the backbone of modern Intrusion Detection Systems (IDS). Before the advent of deep learning, classical ML models were used to classify network traffic as benign or malicious. These foundational algorithms are important as they serve as a baseline for assessing more advanced systems such as the federated MLP in ACTIS. Hence, it is still very important to understand them.

2.4.1 Logistic Regression

Logistic Regression is a basic supervised learning technique that is commonly used to solve binary and multi-class classification problems. Despite its mathematical simplicity, it was popular in early network intrusion research due to its ease of understanding and low computational requirements. It calculates the likelihood of a given data flow belonging to a certain attack type, such as Distributed Denial of Service or botnet activity. This calculation utilizes a logistic sigmoid function, which generates a prediction from a linear combination of input features. In the context of network security, Logistic Regression can be effective when there exists a distinct linear relationship between features for example, total packets and flow duration—and the outcome being predicted. However, Logistic Regression has difficulty handling the subtle non-linear behaviors of modern attacks, since it cannot model the complex interactions between features. This results in poor performance in the more sophisticated attacks, especially in attacks that manipulate packet headers to simulate normal traffic, since the decision boundary learned by Logistic Regression is too simple to capture such complex behaviors.

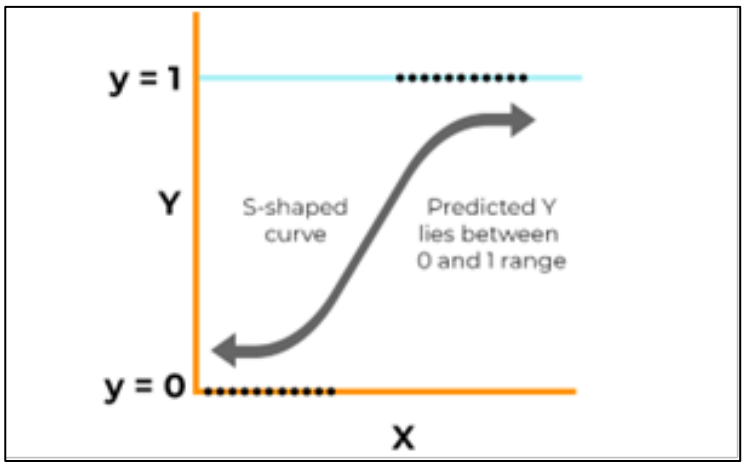

Fig 2.4 Logistic Regression curve

2.4.2 Random Forest Classifier

The Random Forest Classifier is superior to linear models because it builds multiple decision trees during training and makes the final prediction based on the majority vote of these trees. The training process for each tree involves random sampling of the data and features, known as bagging. This approach significantly mitigates overfitting, a common problem in network security due to the presence of noise or redundancy in data. Therefore, Random Forests are highly regarded in intrusion detection.

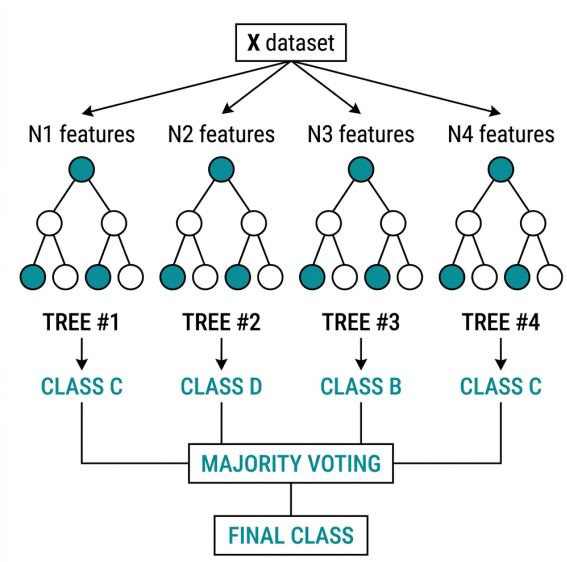

Fig 2.5 Random Forest Classifier

2.4.3 Support Vector Machines (SVM)

Support Vector Machines are awesome as powerful classifiers. They map data points into a high dimensional space and find the best line (well, hyperplane) to separate different classes as distinctly as possible. This makes them extremely useful in network security to differentiate safe traffic from harmful attacks. Using special tools such as the Radial Basis Function kernel, Support Vector Machines map data into even higher dimensions. This allows them to handle super complex boundaries that simpler models can't. In testing these machines exhibit excellent accuracy when applied to well curated datasets. However, their application in actual networks is difficult. Live traffic doesn't behave, there's a lot more normal data coming in than strange attack data, making it difficult to maintain that precision in real life.

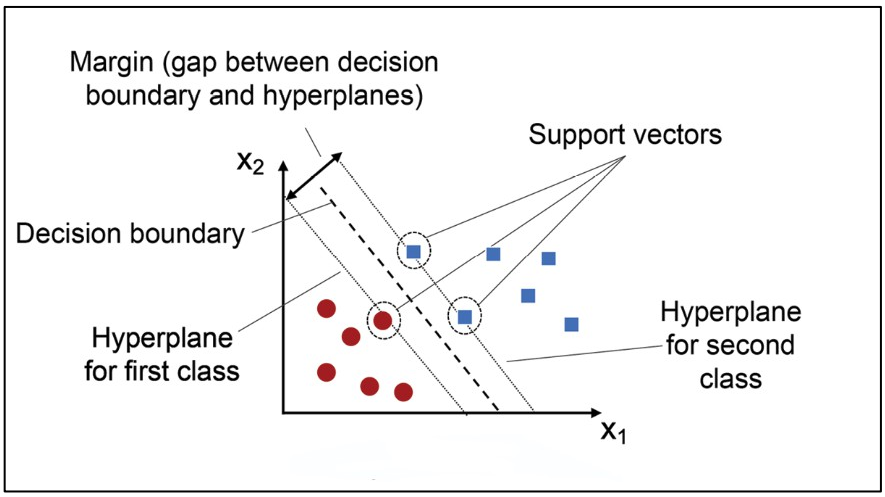

Fig 2.6 Support Vector Machine graph

2.4.4 Naive Bayes

The Naive Bayes classifier is a super efficient probabilistic machine learning algorithm which uses Bayes' theorem. It determines the probability of a hypothesis given prior knowledge. This method assumes that all features in the input are conditionally independent of each other given the class label.

Although this assumption is very far from reality for complex systems, Naive Bayes performs well in network anomaly detection and its use is justified by its fast computational performance and inference time, making it ideal for resource-constrained devices in edge computing scenarios.

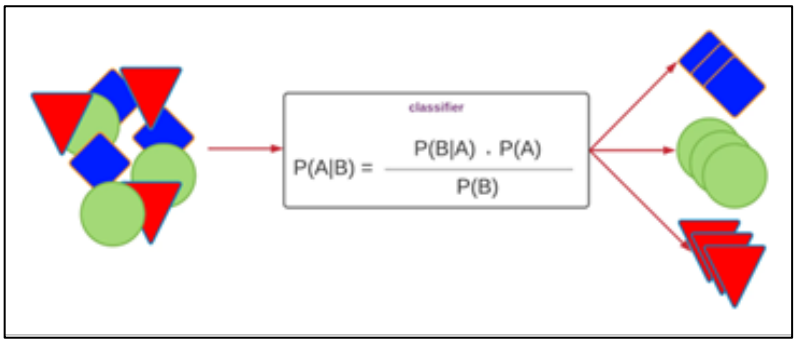

Fig 2.7 Naive Bayes Classifier

2.4.5 Comparative Insights

ACTIS deep learning architecture, particularly the deep Multilayer Perceptron, overcomes all these old limitations effectively. It utilizes many layers of non-linear activation functions to learn complex patterns directly from the raw NetFlow vectors, without any manual feature engineering. Most importantly, the Multilayer Perceptron relies on continuous math gradients through backpropagation, so its numerical weights can be safely shipped around, managed by FedProx, and seamlessly aggregated at a central server.

2.5 Feature Engineering for Network Intrusion Detection

ACTIS relies on transforming the raw NetFlow v2 data to detect network intrusions. This transformation process converts the basic information into useful forms that can be learned by the models. The raw data from different datasets, such as NF-CSE-CIC-IDS2018-v2 for Enterprises and NF-BoT-IoT-v2 for IoTs, undergoes a thorough transformation.

- Basic TCP Flag Decomposition looks at individual binary features from the TCP flags field like SYN, ACK, FIN, RST, PSH, and URG. This helps the model spot attacks, including SYN floods and RST injections.
- Linguistic Window Ratio Analysis compares TCP window sizes in forward and backward flows. This ratio points out asymmetric communication

patterns that are common in command-and-control channels.

- Next Byte-per-Packet Ratios calculate the total bytes to total packets for both directions. These ratios help find abnormalities in volume that suggest DDoS amplification attacks.
- Flow Rate Features measure packets-per-second and bytes-per-second. They tell us how intense network flows are over time.
- Packet Size Variance is figured out by finding the gap between max and min packet lengths within a flow. Usually, benign apps show high variance, but botnet beacons or ping floods typically send packets of uniform size, leading to almost zero variance.
- Lastly Retransmission Ratios track how many times bytes or packets are sent again compared to the total amount. When this ratio is too high, it might show network congestion, caused either by a volumetric attack or interference from an inline intrusion prevention system.
- Protocol One-Hot Encoding: The first protocol uses binary flags to show whether the transport is TCP, UDP, or ICMP. This lets the deep learning model adjust to the protocol type, since UDP floods aren't like TCP-based attacks.
- Log-normalized flow duration comes next. It logs the total flow time to stop super long durations, such as persistent C2 connections, from overly affecting gradient updates. Flow times vary widely—from milliseconds to hours and this tweak balances things out.

2.6 Federated Network-Specific Challenges and Considerations

Using a collaborative intrusion detection system across different organizations is tough for reasons beyond what traditional machine learning faces:

1. Data Heterogeneity (Non-IID Distributions): First, the types of network attacks observed differ drastically from one organization to another. For example, enterprise networks may see web app attacks and brute force intrusions, while IoT networks may see DDoS floods and botnet activity.

2. Communication overhead and latency: Then there's the big issue of communicating all that info between locations. Sending updates from one place to another can really eat up bandwidth and take time, especially when these locations are spread out. This lag could actually endanger systems since new threats might hit before updates can protect against them.
3. Adversarial Participants and Model Poisoning: Finally, not all nodes can be trusted. Some may even be trying to sabotage the system, submitting malicious updates that compromise overall security. The ACTIS tackles this with a trust-aware approach. It monitors for suspicious activity and penalizes nodes behaving strangely to prevent any tampering.
4. Privacy Leakage from Model Updates Even without sharing the raw data, previous works have demonstrated that gradient and weight updates can be inverted via model inversion and membership inference attacks to reconstruct private training samples. ACTIS addresses this issue by adding Differential Privacy noise to the shared threat intelligence embeddings and model weight updates.
5. Concept Drift and Evolving Threat Landscapes: Attack vectors change rapidly, posing a challenge for models trained on outdated data. As adversaries evolve, ACTIS leverages a continuous federated learning cycle along with the RAG-based Immunity Engine to capture new threat intelligence in real-time. This synergy ensures the model remains current.

2.7 Evaluation Metrics

Different evaluation metrics are used to measure the effectiveness and reliability of attack detection technologies. These indicators help in the assessment of the model's ability to distinguish between legitimate and fraudulent instances of attacks. The most commonly used metrics are shown below:

2.7.1 Accuracy

Accuracy is the ratio of correctly predicted observations to the total observations.

Formula:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- **TP (True Positives)**: Fake attacks correctly identified as fake.
- **TN (True Negatives)**: Real attacks correctly identified as real.
- **FP (False Positives)**: Real attacks incorrectly identified as fake.
- **FN (False Negatives)**: Fake attacks incorrectly identified as real.

Use Case: Accuracy is a good metric when the dataset is balanced, i.e., the number of fake and real attacks samples is nearly equal. However, it can be misleading when the dataset is imbalanced.

2.7.2 Precision

The ratio of accurately anticipated positive observations is known as precision (true fake attacks) to the total predicted positive observations.

Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Use Case: In circumstances when false positives are prevalent, accuracy is crucial **costly**, such as labeling real attacks as fake, which could damage reputations or spread distrust.

2.7.3 Recall (Sensitivity or True Positive Rate)

Recall is the proportion of accurately anticipated positive observations to all actual positive observations is known as recall.

Formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Use Case: Recall is crucial when the cost of missing actual fake attacks (false negatives) is high. For example, failing to flag fake attacks could have serious consequences.

2.7.4. F1-Score

The F1-score is the harmonic mean of precision and recall. It gives a single score that balances both concerns.

Formula:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Use Case: F1-score is particularly useful when the dataset is imbalanced**,** and we want to balance the trade-off between precision and recall.

2.7.5. Support

Support refers to the number of actual occurrences of each class in the dataset.

Formula: Support = Number of actual cases for every class (no mathematical expression as it's a raw count).

Use Case: Support helps understand class distribution and is crucial in evaluating classifier performance on imbalanced datasets.

2.7.6. ROC-AUC Score (Receiver Operating Characteristic – Area Under Curve)

The ROC curve plots the True Positive Rate (Recall) against the False Positive Rate (FPR). The AUC (Area Under the Curve) symbolizes the model's capacity for class distinction.

Interpretation:

- **AUC = 1.0** → Perfect classifier

- **AUC = 0.5** → No discrimination (random guessing)
- **AUC < 0.5** → Worse than random

Use Case: ROC-AUC is valuable for comparing models and is less sensitive to class imbalance. It gives insight into the ranking quality of predictions.

2.8 Summary

This chapter reviewed in detail the theoretical background and main concepts behind the ACTIS framework. The evolution of Intrusion Detection Systems was discussed starting from isolated and signature based methods to collaborative approaches using machine learning. The fundamentals of Federated Learning were reviewed and the challenges of non-IID data distribution that FedProx algorithms address in the context of different networks were discussed. While classical machine learning models are established benchmarkers, deep learning methods such as the Multilayer Perceptron are able to model complex non-linear relationships found in modern network traffic.

The text also highlighted the importance of feature engineering to transform raw NetFlow data to useful features for detection. In addition, it highlighted challenges in federated, multi-organization security settings, such as the possibility of hackers in the group, communication costs, and the evolution of threats. Moreover, Large Language Models were presented for the automatic removal of personal information and the secure sharing of threat intelligence through a combination of Retrieval-Augmented Generation and Differential Privacy. Finally, a rigorous evaluation framework with metrics including accuracy, precision, recall, F1-Score, support, and ROC-AUC, was presented to set the stage for the rigorous evaluations presented in the latter parts of the work.

Chapter 3

SOFTWARE REQUIREMENT SPECIFICATION OF ACTIS

The Agentic Collaborative Threat Intelligence System (ACTIS) is a stand-alone cyber security framework for privacy protection in distributed networks of multiple organizations. ACTIS is not an extension to a commercial SIEM platform but an independent framework, which processes independently from the intake of raw NetFlow data and the training of federated models to the generation of secure threat information and the deployment of perimeters.

3.1 Product Perspective

From a product perspective, ACTIS is a combination of three traditionally separate areas: network intrusion detection, federated machine learning, and generative AI automation. Each org within the system has its own private node, which only communicates data through a central server and a shared database. So, raw network traffic never leaves the organization's perimeter, keeping the data secure within the organization.

- ACTIS consumes NetFlow v2 data in the Parquet format and uses two datasets for analysis. These datasets are NF-CSE-CIC-IDS2018-v2 and NF-BoT-IoT-v2. The first is related to enterprise traffic, which includes DDoS, DoS, brute force, web attacks, and botnets. The second is related to IoT sensor traffic, such as DDoS/UDP, DoS/TCP, reconnaissance, and theft. For real-time data, ACTIS can consume live feeds from CSV files, log tailing, and nfdump pipe input.
- The Agentic Privacy Engine uses the Google Gemini 2.0 Flash API for contextual analysis and MITRE ATT&CK mapping. In case of an issue, it falls back to the Groq-hosted LLaMA 3.1 70B model for backup and smooth operation.

- Threat data is stored in a ChromaDB vector store, and private data is stored in three different collections: threat_summaries, defense_patterns, and mitre_reference. Threat updates and reports are transferred using ZeroMQ (ZMQ) inter-process communication, which enables local nodes to communicate with the central server on specific ports. The default port for the FL server is 5555 and 5556 for the dashboard.

- Metrics are used to evaluate the performance of the model. They aid in determining effectiveness by producing a confusion matrix and ROC curve that simplify the visualization of classification performance. Created with flexibility in mind, the system integrates seamlessly into web-based dashboards, content moderation tools, and mobile platforms. Its lightweight design enables it to adapt well to different environments.

3.2 Specific Requirements

The system should meet the comprehensive and specific functional and non-functional requirements to satisfy the stakeholders' needs. These requirements ensure that the fake news detection system operates accurately, efficiently, and reliably, fulfilling the overarching goals of misinformation detection and content validation.

3.2.1 Functional Requirements

The functional requirements spell out the key tasks the system needs to handle for accurately spotting deceitful content:

- Data Set Details: For the ACTIS intrusion detection system, the datasets come from the publicly accessible NF-CSE-CIC-IDS2018-v2 and NF-BoT-IoT-v2 on Kaggle. These are widely used in studies and practical work on network security, plus include some synthetic data from the Synchronet setup. The Enterprise dataset alone boasts over 18 million NetFlow entries, showcasing typical enterprise network traffic and covering DDoS, DoS, Brute Force, Web Attacks, and Botnet events. Meanwhile, the IoT dataset includes around 37 million entries and deals with sensor network

traffic, touching on DDoS/UDP, DoS/TCP, Reconnaissance, and Theft attacks. Raw data is transformed into a 41-dimensional numeric feature vector, and then 25 extra features get added, resulting in a final 66-dimensional input format. After this, the targets are sorted into a single taxonomy of nine attack classes plus one for normal activity. It's all coded with integers to streamline the process for the multi-class classifier.

- Collection and Processing: The system shall be able to ingest large-scale NetFlow telemetry datasets in Parquet format from configured dataset directories. It shall perform preprocessing operations such as schema normalization (from 43 columns to 41 numeric features), missing value imputation, extreme value clipping, and feature scaling using StandardScaler normalization.
- Model Training and Federated Aggregation: The system shall train a PyTorch-based Multilayer Perceptron (MLP) IDS with a hidden layer architecture of neurons, ReLU activations, and a dropout rate of 0.3 at each local node. It shall support the FedProx proximal regularization term in the local loss function to prevent weight divergence on non-IID data.
- Threat Analysis and PII Sanitization: When a trained local IDS detects an anomalous network flow, the Agentic Privacy Engine uses the Gemini 2.0 Flash LLM in conjunction with LangChain to perform a structured threat analysis, mapping the anomaly to specific MITRE ATT&CK tactics and techniques. The Agent Sanitizer removes all Personally Identifiable Information from the report, such as IP addresses, MAC addresses, hostnames, credentials, and email addresses, using a combination of LLM-driven contextual understanding and strict regex pattern matching.
- Evaluation and Validation: The evaluation measures the accuracy, precision, and a weighted F1-score to evaluate the performance of each federated node as well as the global model independently. Confusion matrices are also generated to identify issues and detection trends within each attack class.

3.2.2 Performance Requirements

The performance requirements specify the desired accuracy, efficiency, and adaptability of the intrusion detection system:

- **Classification Latency:** The local MLP model shall classify a single network flow in under 5 milliseconds on CPU hardware to support real-time monitoring pipelines.
- **Federated Convergence:** The global model shall achieve convergence (defined as less than 1% accuracy change between consecutive rounds) within 10 federated training rounds.
- **PII Sanitization Throughput**: The Agentic Privacy Engine shall process and sanitize a threat report within 10 seconds per invocation, including the LLM API call and regex validation loop.
- **RAG Retrieval Latency**: Semantic similarity searches against the ChromaDB vector store shall return results within 500 milliseconds for a database containing up to 10,000 threat embeddings.
- **Zero-Day Detection Rate**: The per-protocol autoencoders shall achieve a minimum detection rate of 95% for previously unseen DDoS attack variants.
- **Scalability**: The federated architecture shall support up to 10 concurrent client nodes without significant degradation in aggregation time.

3.2.3 Software Requirements

The software requirements spell out what you need in terms of tools, libraries, frameworks, and dev environments for designing, implementing, and rolling out the intrusion detection system:

- Operating System: First up, the system is going to be based on Windows, Linux (Ubuntu), and macOS, giving you a wide range of OS choices whether you're developing or deploying stuff. Next, we're sticking with PyTorch for building and tuning our Multilayer Perceptron (MLP) IDS model and a special FedProx algorithm.

- Programming Language: Python is the language of choice along with a bunch of useful libraries: NumPy and Pandas for data, Scikit-learn for basic models and performance evaluation, while Matplotlib and Seaborn for visualizing results. Plus, we're using Flower for federated learning set-ups and LangChain along with ChromaDB for intelligent AI workflows and info storage.
- For writing code, our preferred environments are modern IDEs like Google Colab, Jupyter Notebook, and Visual Studio Code. They're excellent because they allow interactive development and even speed up things with GPUs.

3.2.4 Hardware Requirements

Moving onto hardware needs, the kit for ACTIS needs to match the heavy lifting required by distributed deep learning, live data handling, and the RAG setup:

- Processor: First, you'll want an Intel Core i7 from at least the 10th gen, AMD Ryzen 7, or something similar from Apple, like the M1. This ensures that your processor isn't the bottleneck when it comes to dealing with data, running models locally, and other computing duties.
- Second, 16GB RAM is a must-have to juggle big data loads, but 32GB or more would be awesome to keep everything running smoothly.
- Now, you definitely need a dedicated GPU – an NVIDIA RTX 3060 or 3080 fits the bill here – as it'll supercharge those neural net trainings and the RAG pipeline run from the main server.
- Storage-wise, aim for 1TB of SSD to house all your logs, model checkpoints, training records, and vector DB files.
- Last thing? You'll also require a fast internet connection – minimum of 100 Mbps for grabbing pre-trained SentenceTransformer models, sending out fed learning stuff, and swiftly connecting to the Gemini 2.0 platform.

3.2.5 Design Constraints

- Privacy Compliance: The system architecture must ensure that no raw network telemetry or Personally Identifiable Information is ever transferred

outside the organizational boundary. All communication between nodes must be limited to aggregated model weights, differentially private embeddings, and sanitized threat reports. This is a non-negotiable requirement and takes precedence over all performance optimization considerations.

- API Rate Limits: The Gemini 2.0 Flash API has rate limits on the number of requests per minute. The Agentic Privacy Engine must have appropriate backoff and retry logic to operate within these limits. A fallback to the Groq-hosted LLaMA 3.1 70B model is available for high-volume use.
- Dataset Licensing: The NF-CSE-CIC-IDS2018-v2 and NF-BoT-IoT-v2 datasets are obtained from Kaggle and are subject to their respective licensing terms. The system must not redistribute raw dataset files.
- Non-IID Data Distribution: The federated training process must assume that data distributions are inherently heterogeneous (non-IID) across the different nodes. The system cannot impose data redistribution or sampling strategies that require access to other nodes data.
- Deterministic Sanitization: The LLM temperature for the Agentic Privacy Engine is capped at 0.1 to generate near-deterministic outputs for security-critical. The use of higher temperature values that inject creativity or variation is explicitly forbidden in this context.
- Offline Operation: Once the initial model and datasets are downloaded, the core federated training and classification pipeline must be able to function without an active internet connection. Only the LLM-based threat analysis and RAG intelligence sharing components require external API access.

3.3 Summary

This chapter provides a comprehensive Software Requirement Specification for the ACTIS framework, linking cybersecurity theory and practical software engineering. It introduces ACTIS as a modular, distributed system, rather than a single application, incorporating Federated Learning, Generative AI, and Network Intrusion Detection. This design ensures the adaptability, hardware-agnostic nature and scalability of ACTIS. The specification covers the entire system lifecycle,

exploring nine critical modules. Data flows through this architecture from acquisition and preparation to deep learning analysis and reliable integration. The process includes safer threat making, applying privacy rules, and smart generation methods to create defenses against attacks. They also set tough performance goals, providing clear metrics for the system's success. These guidelines focus on how quickly edges should identify threats, how fast global parts need to sync up, and how nimbly they must handle big info loads. This all helps make sure ACTIS is ready to stop dangerous assaults in real time.

Chapter 4

HIGH-LEVEL DESIGN SPECIFICATION OF THE ACTIS FRAMEWORK

To ensure its resilience, privacy standards compliance, and operational efficiency across different networks, the Agentic Collaborative Threat Intelligence System (ACTIS) has been designed based on basic rules of architectural design.

4.1 Design Consideration

Designing for privacy is paramount; it's not an afterthought; the assumption is that it's baked into all aspects of the system. Raw network data never leaves the organization. Only three types of information are shared between groups: first, non-traceable model updates; second, private data about meanings; and third, sanitized threat reports with no identifying information. These rules must be followed rigorously from start to finish.

4.1.1 General Considerations

ACTIS can be split into separate pieces that can be tested independently. These pieces include the data pipeline, local operations, central server, test suite and live monitoring. This division allows developers to work, test and replace parts of the system without affecting the rest of the system.

4.1.2 Development Methods

Finally ACTIS works with an iterative development and verification approach, in line with the project's development plan. They use a nine-step verification process, ensuring that each new part is verified before moving on to the next:

- Phase Driven Development: The project adopts Phase-Driven Development, which consists of nine validation phases. These phases are data pipeline, local intrusion detection system (IDS), federated training, privacy checks, zero-day detection, RAG intelligence, report quality, replication of the baseline model, and full evaluation. Each phase involves the completion of the respective validation script's preset standards. They build the system layer by layer, starting with the basic pieces and moving to the more complex ones. First, developers work on the data pipeline and feature engineering modules, then the local IDS model and federated aggregation layer. They also look at privacy and sanitization engines, finishing with the RAG and immunity systems. Each layer gets tested on its own before being combined**.**
- They also test thoroughly, using Pytest for individual parts and end-to-end checks with their validation suite. So each piece has exposed interfaces that make isolated tests easy.
- They have a Typer based CLI (cli.py) to run everything from fetching data, to federated training and live watching. It allows them to automate and control the entire process with simple commands.

4.2 Architectural Strategies

The ACTIS framework employs carefully tuned hyperparameters across multiple subsystems, each calibrated through empirical experimentation on the target datasets:

4.2.1 Hyper Parameter Tuning

Tuning of the hyperparameters is crucial for optimizing model performance. For the BERT-based classifier, key parameters include:

- Number of federated rounds: 10
- Number of client nodes: 4
- Local training epochs per round: 5 (balancing local overfitting vs. underfitting)
- Batch size: 64 (optimized for memory efficiency on consumer hardware)

- Learning rate: 1e-3 (Adam optimizer with adaptive moment estimation)
- Trust epsilon: 1e-6 (numerical stability constant in trust score computation)
- Hidden layer architecture: [128, 64, 32] neurons

- Dropout rate: 0.3 (regularization to prevent co-adaptation of neurons)
- Input dimension: 41 base features + 25 derived features = 66 total

4.2.2 Feature Engineering

ACTIS uses a feature engineering method based on real analysis of the differences between the attack types in their datasets. They extend the initial 41-dimensional NetFlow feature vector with 25 extra features divided into two groups. The added features deal with volume, direction and timing of network traffic. They look at things such as inbound vs outbound byte to packet ratios, packet size comparisons, symmetry levels, etc. Other features track throughput, flow times, bytes and packets rates, packet size variances and protocol hotspots – think TCP, UDP, and ICMP. They also track retransmission rates and bursts for both directions, and established port flags.

4.2.3 Scalability and Adaptability

Scalability: ACTIS is designed to scale with a simple mechanism for adding new organization nodes. Just plug in the new node's data, its share of the partition, and basic info using the COMPANY_CONFIG in config.py file.

Adaptability: For adaptability, the setup uses Parquet files with columnar storage via PyArrow for efficient data processing, even with datasets larger than RAM capacity. ChromaDB offers durable disk-based indexing for threat embeddings, enabling the system to scale to millions of these without needing everything to be loaded into memory.

4.2.4 Evaluation Metrics

For performance evaluation, ACTIS validates the accuracy, precision, recall, weighted F1-score, and ROC-AUC per node and global. They also build confusion matrices, for all nine attack classes and normal traffic, to identify any consistent errors. Finally, they ensure no personal information leakage by cross-validating shared threat reports with regexes for IPs, MACs, emails, hosts, and

user credentials, aiming for no leaks across the board.

4.3 System Architecture for ACTIS Framework

The Fig 4.1 represents the architecture of the ACTIS:

1. Input Layer: Consumes unprocessed NetFlow v2 telemetry data from Parquet datasets (CIC-IDS2018 for Enterprise, BoT-IoT for IoT) or real-time streams from CSV files, log tailing, or nfdump pipes.
2. Preprocessing Module: Converts raw 43-column schema to 41-dimensional numeric feature vector, applies StandardScaler normalization, computes inverse-frequency class weights for class imbalance, distributes data across 4 organizational nodes to simulate non-IID federated conditions.
3. Federated Aggregation: Sends local model weights from all 4 nodes to the central server, which computes dynamic trust scores based on training loss and performs Trust-Aware FedAvg aggregation.

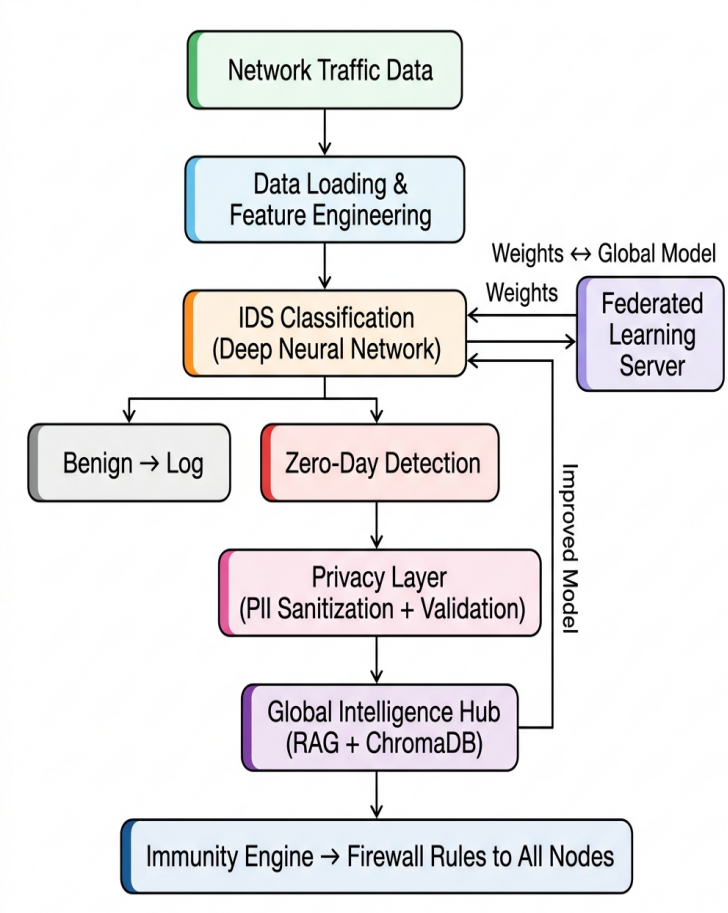

Fig 4.1 System Architecture

4. Privacy & Sanitization Layer: Upon detection of an anomaly, the Gemini 2.0 LLM assesses the threat and maps it to the MITRE ATT&CK tactics. The Agent Sanitizer removes all PII (IPs, MACs, credentials) using a rigorous regex retry loop that ensures 0% leakage. The sanitized report is converted into a 384-dim semantic embedding with Differential Privacy noise added.

4.4 Data Flow Diagrams

This section describes the flow of data through various components of the intrusion detection system. It helps to visualize the way data is collected, processed and transformed into meaningful predictions. We decompose the system with increasing details at each level of Data Flow Diagram (DFD) to improve understanding and traceability of operations.

4.4.1 Data Flow Diagram Level 0

Figure 4.2 depicts the Level 0 DFD that captures the topmost level of abstraction of the ACTIS framework as a single central process and demonstrates how it interacts with four external entities. The main source of input is the Network Infrastructure, which provides raw network traffic data to the ACTIS system for analysis.

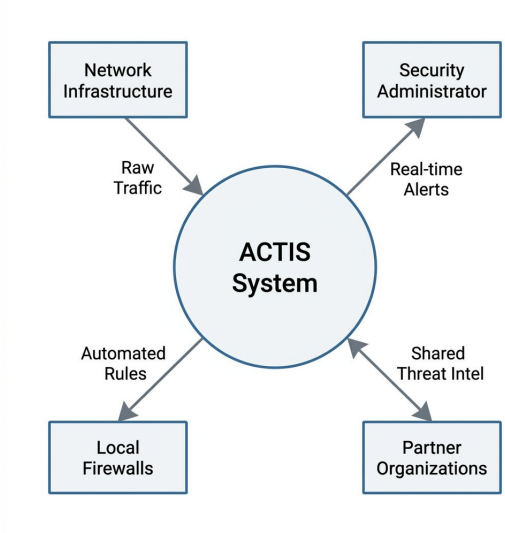

Fig 4.2 DFD Level 0 diagram

4.4.2 Data Flow Diagram Level 1

The Fig 4.3 shows Level 1 DFD decomposes the ACTIS system into four

principal processes that are linked in a sequential pipeline. The first process Data Engineering is responsible for ingesting raw NetFlow telemetry, and normalizing the 43-column schema into a 41-dimensional feature vector.

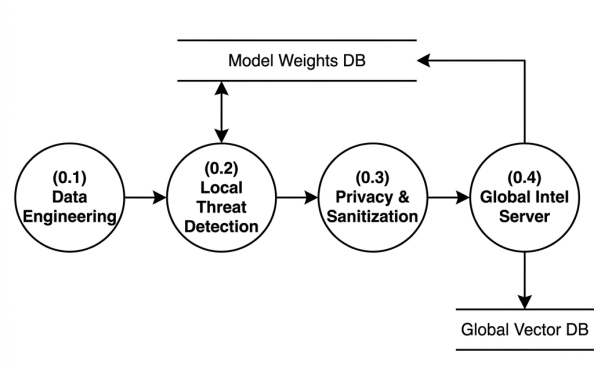

Fig 4.3 DFD Level 1 diagram

4.4.3 Data Flow Diagram Level 2 for Tokenization Module

The Fig 4.4 represents a DFD provides the most granular decomposition, breaking down the internal operations of a single ACTIS Local Node into five interconnected sub-processes. Sub-process 1.1, Feature Engineering, receives raw data from external sensors and logs, extracts network and host-level metrics, and produces the 66-dimensional feature vector that serves as the input representation for all downstream analysis.

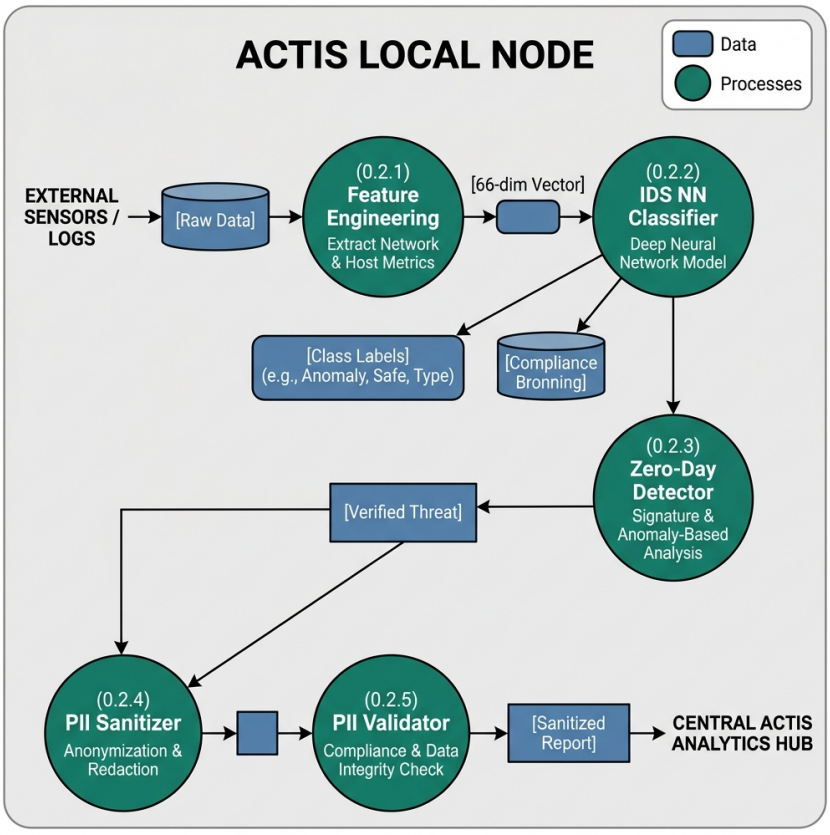

Fig 4.4 DFD Level 2 diagram for Tokenization

4.4.4 Data Flow Diagram Level 2 for Feature Engineering Data Module

Fig 4.5 illustrates the feature engineering process, which transforms raw NetFlow telemetry into numerical representations suitable for the classification model.

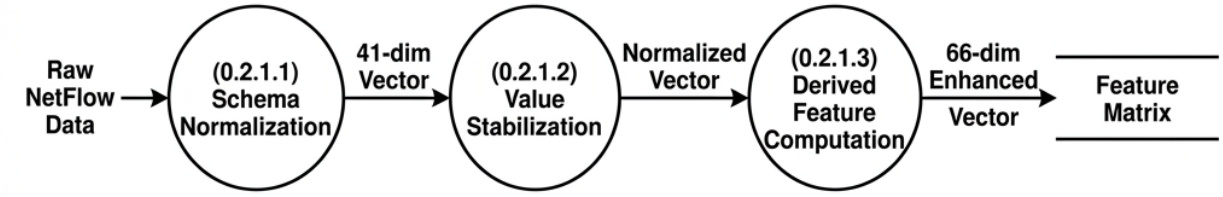

Fig 4.5 DFD Level 2 diagram for Feature Extraction

4.4.5 Data Flow Diagram Level 2 for Trained Data Module

Fig 4.6 illustrates the model training pipeline where the enhanced feature matrix is fed to the PyTorch MLP classifier. It depicts the FedProx training loop through proximal regularization of 5 local epochs, iterative weight updates, and parameter optimization, in comparison to the baseline of the global model, with outputs like updated local model weights and training loss metrics being transmitted to the FL server.

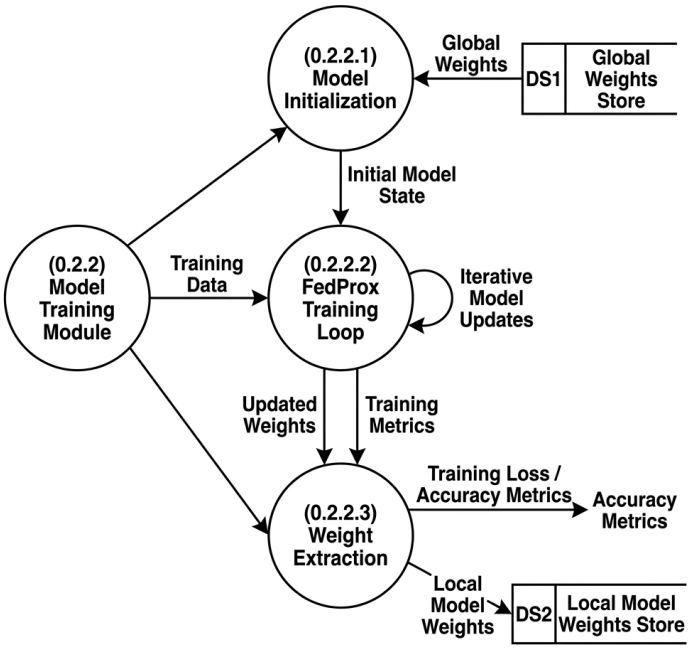

Fig 4.6 DFD Level 2 diagram for Trained Model

4.4.6 Data Flow Diagram Level 2 for Classifier Data Module

The Fig 4.7 represents the classification and detection phase using the trained MLP. It details how the model receives the 66-dimensional feature vector.

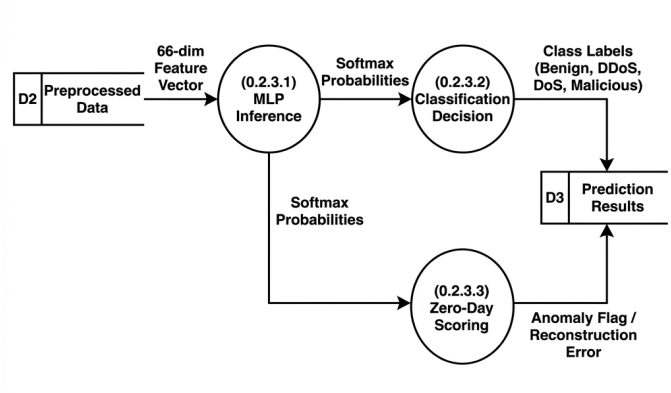

Fig 4.7 DFD Level 2 diagram for Classifier Data Module

4.4.7 Data Flow Diagram Level 2 for Evaluation module

Fig 4.8 delves into the Evaluation Module, which assesses the performance of the trained ACTIS intrusion detection model. Once the model makes predictions on the test data.

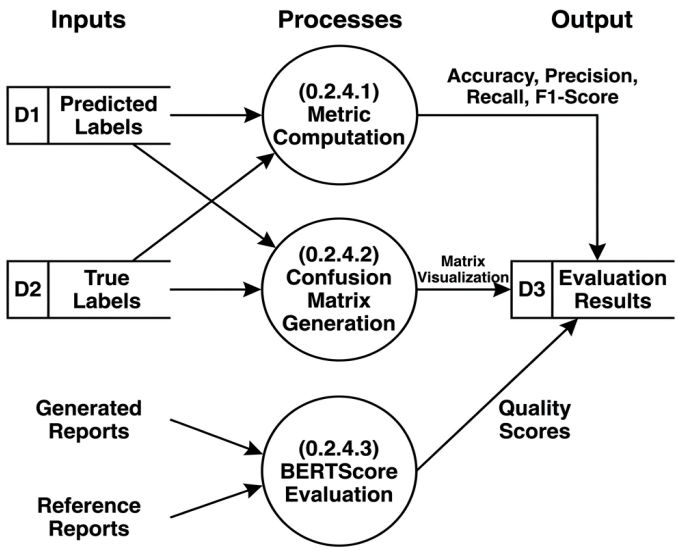

Fig 4.8 DFD Level 2 diagram for Evaluation module

4.5 Summary

The system's high-level design employs a modular, scalable federated architecture using a PyTorch MLP model with hyperparameter tuning and 66-dimensional feature engineering for flexibility. The three-tier Data Flow Diagram illustrates the process from an overall context view (Level 0) to core modules (Level 1)

and detailed sub-processes (Level 2) on feature engineering.

Chapter 5

DETAILED DESIGN OF ACTIS

This chapter describes the internal design of ACTIS and the decomposition into modules. The system consists of several connected parts, each one performing a specific task along the pipeline. Such architecture ensures that all the components can be tested, scaled and reused independently. Moreover, the use of IDS provides a rich context for effective attack classification.

5.1 Structure Chart

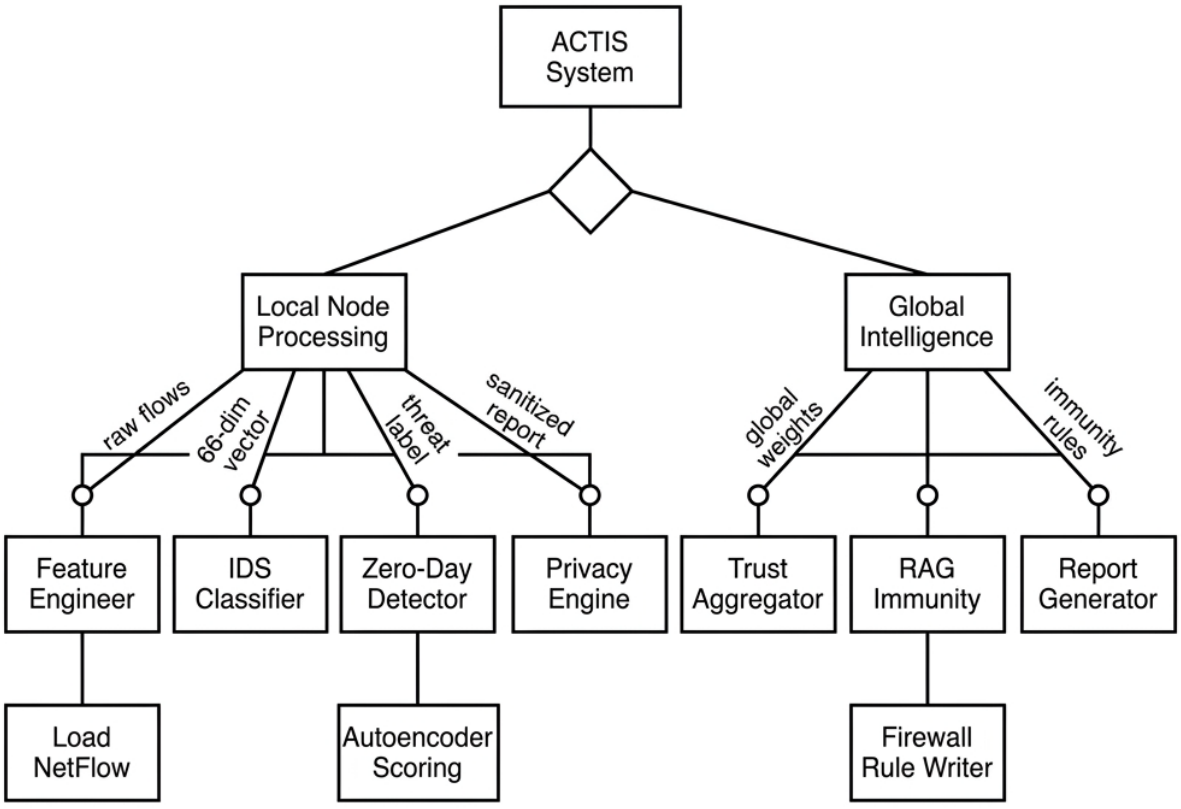

Fig 5.1 Structure Chart

Figure 5.1 shows the detailed structural diagram of the ACTIS system, breaking the architecture down into its four main packages and their modules. At the highest level, ACTIS is divided into four primary packages: Data Pipeline (src/data), Local Detection Node (src/local_node), Global Server (src/server), and Evaluation Suite (src/evaluation). Each of these packages consists of sets of modules that work together with well-defined responsibilities.

For the Data Pipeline package, this includes the ingestion, normalization, feature engineering, MITRE mapping, and summarization. The Local Detection Node package includes the MLP model definition, FedProx training, zero-day detection, threat analysis with LLMs, PII sanitization and validation, differential privacy, and node orchestration.

5.2 Module Design

In Section 5.2, we get into more detail about the app's functional modules, explaining inputs, internal processing, outputs, and the tools used for each one, including spec tables.

5.2.1 Data Input and Collection Module

This section is about the Data Input and Collection Module. This module takes raw network telemetry and processes it for further use. It uses the FedIntelDataLoader class implemented in loader.py. This class reads NetFlow v2 data from dataset directories, which are in Apache Parquet format. It works with two benchmark datasets namely NF-CSE-CIC-IDS2018-v2 for enterprise traffic and NF-BoT-IoT-v2 for IoT sensor traffic.

Table 5.1 Data Input and Collection Module

Parameter	Description
Input	Raw NetFlow v2 telemetry in Parquet format (.parquet)
Output	Partitioned feature matrix (X) and target labels (y)
Libraries Used	Pandas, PyArrow, pathlib
Edge Case Handling	Missing Parquet files, invalid partition index, empty datasets

5.2.2 Data Preprocessing module

The Data Preprocessing Module converts the raw ingested data into a clean, normalized form suitable for model training. The preprocessor.py module implements the Preprocessor class that performs various important operations. First, it reduces the raw 43-column NetFlow schema into a 41-dimensional numeric feature vector by dropping non-numeric fields.

Table 5.2 Data Preprocessing module

Parameter	Description
Input	Raw 43-column NetFlow feature matrix and labels
Output	Normalized 41-dimensional feature vector with class weights
Libraries Used	Scikit-learn (StandardScaler), NumPy, Pandas
Edge Case Handling	NaN/Inf values, zero-variance columns, extreme class imbalance

5.2.3 Feature Engineering Module

The Feature Engineering Module enhances the discriminative power of the input representation by extracting additional features from the base vector. The feature_engineer.py module implements the Feature Engineer class, which computes 25 derived features, divided into two categories. The first category comprises 17 temporal and byte-level features such as bytes-per-packet ratios, packet and byte asymmetry indices, throughput ratios.

Table 5.3 Feature Extraction Module Specification

Parameter	BERT Embeddings
Input	Sanitized threat report (PII-free text)
Output	384-dimensional dense semantic vector
Dimensionality	Fixed (384)
Semantic Context	Yes
Differential Privacy	Gaussian noise injected (ε=1.0, σ=0.1)
Model Used	all-MiniLM-L6-v2

5.2.4 Classification Module

The Classification and Detection Module is the core intelligence of each local node, responsible of the detection of known and unknown attacks. The ids_model.py module contains the class IDSModel, a Multilayer Perceptron based

on PyTorch with a hidden layer architecture of [128, 64, 32] neurons, ReLU activations and a dropout rate of 0.3. The ids_trainer.py module contains the class IDSTrainer which implements the FedProx training loop, running 5 local epochs per federated round with Adam optimizer with a learning rate of 1e-3 and a batch size of 64.

Table 5.4 Classification Module

Parameter	Description
Input	66-dimensional enhanced feature vector
Output	Class labels (9 attack types + Benign), zero-day anomaly flags, sanitized threat reports
Libraries Used	PyTorch, LangChain, langchain-google-genai, SentenceTransformers, ChromaDB

5.2.5 Prediction and Evaluation Module

The last module is Prediction and Evaluation as in Table 5.5 is responsible for making the predictions and evaluating the performance of the model. The projected probabilities are converted into binary labels using a threshold (usually 0.5). The performance indicators such as precision, accuracy, recall, and F1-score are calculated by comparing the predictions with the actual labels. Moreover, ROC curves and confusion matrices are generated to demonstrate the efficacy of the classifier. These metrics are calculated and visualized using libraries such as Scikit-learn and Seaborn.

Table 5.5 Evaluation Module

Metric	Description
Accuracy	Proportion of correctly classified network flows out of total flows
Precision	Ratio of true positives to all predicted positives per attack class
BERTScore F1	Semantic similarity between LLM-generated threat reports and ground-truth references
PII Leakage Rate	Percentage of shared threat reports containing detectable personally identifiable information
Zero-Day Detection Rate	Proportion of previously unseen attacks correctly flagged by per-protocol autoencoders

5.3 Summary

In this chapter, the detailed design of the ACTIS framework is presented. It includes a structure chart and a breakdown at the module level. The structure chart shows the hierarchical composition of the system into four main packages: Data Pipeline, Local Detection Node, Global Server, and Evaluation. Together they constitute 22 connected modules. The specific duties of each module and the details of their implementation are also outlined. For example, the Data Input Module is responsible for Parquet file ingestion and real-time streaming, while the Preprocessing Module is responsible for feature normalization and the class

imbalance problem. The Feature Engineering Module, for example, produces 25 additional discriminative features, making the overall dimensionality equal to 66.

Chapter 6 IMPLEMENTATION OF ACTIS

In this chapter, we talk about how we implemented a fake news detection system (in particular, using the BERT transformer model). We explain our programming language and development environment choices, the main libraries and tools we used, and our training and prediction processes. We also include details on how we tested the system's performance.

6.1 Programming Language Selection

For the language, we wanted something that would work well with machine learning and data handling. For our ACTIS project we decided to stick with Python 3.10+. It just made sense, Python is super popular when it comes to machine learning and cybersecurity. The libraries available are amazing – PyTorch and LangChain for deep learning and NLP, Pandas and NumPy for data juggling. Not only that but its easy-to-use syntax allowed us to quickly prototype things. We also got the added benefit of fast training thanks to GPU support via CUDA.

6.2 Development Environment Selection

For development, we knew we wanted to select tools and an environment that would facilitate experimenting, debugging, and collaboration. Here's what we settled on for that:

- Jupyter Notebook: For interactive development and step-by-step testing of preprocessing, tokenization, and evaluation workflows.
- Google Colab: For initial model training and testing with GPU acceleration. It supports Hugging Face Transformers, Pytorch, and visualizations via libraries such as Matplotlib
- Visual Studio Code: For script development, modular coding, and version-controlled integration of the complete project pipeline.
- GitHub: For version control, collaborative tracking of experiments, and backups of scripts and notebooks.

These environments collectively supported rapid prototyping, scalability, and ease

of access for cloud-based development with hardware acceleration.

6.2.1 Jupyter Notebook

This is a cool tool called Jupyter Notebook, which brings together code execution, documentation, and visuals all in one place. This is great for things like exploring data, trying out different ideas quickly, and tweaking models until you're happy.

6.2.2 Google Colab

Google Colab is a cloud-based Jupyter Notebook environment that provides free access to GPUs and TPUs. It allows easy collaboration, integrates seamlessly with Google Drive and takes care of heavy computations that could be a hassle for your local machine. It is especially useful for testing large models like fine-tuning BERT-based models.

6.2.3 Hugging Face Transformers & Torch

Now if you are looking for the latest in transformer models, like BERT and whatnot, you want to be on Hugging Face Transformers. They also have some pretty handy tokenizers and stuff too. If your goal is to fine-tune BERT for something like fake news detection, this is where you want to be.

6.2.4 Pandas

Pandas takes care of the boring stuff, like moving and cleaning the data. It does the magic with DataFrames to prepare the data, from smashing together CSV files to turning unruly data into neat tables. And then there's NumPy, the guy who handles the number crunching. We're talking vectors, matrices, normalizing data, all that jazz. It handles all the bits and pieces that transformer models need, like those token IDs and attention masks.

6.2.5 Numpy

NumPy is used for efficient numerical operations like vector and matrix operations, normalization, and array-based techniques. It supports the processing of token IDs, attention masks, and other numerical inputs used by transformer models.

6.2.6 Matplotlib & Seaborn

If you want to see how things are going, there's Matplotlib and Seaborn. They can draw curves, heatmaps, and all sorts of plots that show your data and model progress clearly. Want to see ROC curves or check your losses and accuracies? These tools make that easy.

6.2.7 Scikit-Learn

Then there is Scikit-Learn which is really good for the basics. If you are doing logistic regression or random forests, or figuring out how good your model is with precision, recall, F1-scores, etc – Scikit-Learn has you covered. It is also critical for building those confusion matrices.

6.2.8 Pycaret

That's where PyCaret shines, you can try various models to see which works best. It processes the whole thing from start to finish, data prep, model training, model evaluation, in no time at all. Great for obtaining win rates and F1-scores quickly on various methods before committing to one to tweak further.

6.3 Training ACTIS Models

The ACTIS training process is divided into several phases that pass through simulated organizational nodes. There are four of these nodes in total. Here is what happens in each phase:

Phase 1 – Data Preparation: In this phase, the nodes load their specific part of the dataset. Node A and B use the CIC-IDS2018 enterprise traffic, while C and D use BoT-IoT sensor traffic. They start with a 43-column NetFlow layout but cut it down to just 41 features. After standardizing these features using StandardScaler, they add another 25 computed features. So, each entry ends up having 66 dimensions.

Phase 2 – Local FedProx Training: Each node initializes an MLP model with a layer sequence of 128→64→32 and a dropout rate of 0.3. They train the model for 5 epochs with the Adam optimizer, at a learning rate of 1e-3 and a batch size of 64, using the latest global model received from the server. They try to

minimize the cross-entropy loss plus a FedProx term, which ensures that local weights remain close to the global average, aligning the models trained on slightly different data.

Phase 3 – Trust Aware Federated Aggregation: After the local training, the nodes send their updated model weights and training losses to a central server. The server assesses the trustworthiness of each node's model based on the loss, which essentially means that nodes with lower loss are assigned a higher trustworthiness score. It then aggregates the new global model using this trustworthiness and volume-based ratios. This aggregated model is broadcasted to all nodes for another ten rounds of federated training until convergence.

Phase 4 – Zero Day Autoencoder Training Nodes also create separate autoencoders for the TCP, UDP, and ICMP protocols on good traffic alone. These autoencoders are used to identify abnormalities during testing; when these autoencoders do not recognize something, an error occurs above the 95th percentile threshold – which can potentially be a new attack.

Phase 5 – Agentic Intelligence Pipeline: If something funky is detected, the pipeline kicks in. The Agent Analyzer pulls in Gemini 2.0 to determine the attack type, probably talking attack techniques. The Agent Sanitizer then cleans out any personal info using big language models and regex. Validation makes sure no personal info leaks. Before storing embeddings in ChromaDB, some fuzzy Gaussian noise is added to secure privacy. Finally, the Immunity Engine creates firewall rules, querying the database for quick responses and sharing them across the network.

6.4 Summary

This chapter delves into the complete technical setup of the ACTIS framework, turning those theory talks about secure, private cybersecurity into real software. They chose Python 3.10+, sticking with it throughout because of its rich collection of tools, consistency across different platforms, and awesome backing for powerful computing libraries. For keeping things slick and easy to follow, they went with Typer and Rich for command-line duties and loggings on both local nodes and the main server. Network admins get clear views on live updates and safety notices thanks to these. At its core, it's fueled by PyTorch for the deep learning bits – that means, setting up neural networks and doing the calculations fast, aided by CUDA. Each node is also revved up too, speeding through the training tasks without being slowed down by the usual issues in churning data.

On the data side, there's heavy lifting involved, but ACTIS doesn't waste space doing it. Using Pandas and PyArrow, they turned to Parquet files for a speedy way to suck in tons of network info. The shift from those bulky CSVs to Parquet really pays off; way more efficient, especially when reading lots of stuff real quick. After gathering everything, NumPy kicks in to run through all those numbers changing scales, making standards look good, and figuring ratios. Instead of the sluggish way Python would normally go about this, these tools work fast with the help of their backend muscle, letting millions of fields breeze through without holding back.

Chapter 7

SOFTWARE TESTING OF ACTIS

Extensive testing is needed for our collaborative intrusion detection system to be accurate and highly reliable. This is a federated deep learning and agentic AI-based system that will be deployed in real-life where emerging cyber threats can spread extremely fast. We need to test the accuracy, privacy-preserving nature and operational capability of the system in challenging circumstances.

Our tests also go beyond basic validation. We want to show that the system's predictions are reliable even when there is adversarial data. We want to show that it can withstand attacks against its components without breaking. We are also validating that the personal data remains private and that the whole process from data acquisition to threat identification is seamless.

For this, we perform different kinds of testing. We perform unit testing for each part of the system such as pre-processing and feature engineering. We also test the entire pipeline that does the federated learning. Finally, we perform extensive testing of all together using benchmarks and in comparison with other systems like ReGAIN and Tri-LLM.

7.1 Module Testing

In module testing we check that each piece works well on its own. We check how they handle standard tasks and how they react to unusual situations. All these test cases look at inputs, outputs and limits specific to cybersecurity issues.

7.1.1 Test Case for Data Preprocessing Module

Preprocessing of data is the foundation of any machine learning pipeline. In this module, raw NetFlow v2 telemetry is cleaned, normalized, and transformed into a structured format suitable for feature engineering and model input.

Testing Objectives

The goal was to validate that:

- All input NetFlow records are correctly normalized to 41 dimensions.
- No data loss or structural corruption occurs during schema reduction.
- Preprocessing functions handle edge cases (e.g., missing columns, NaN values)

Testing Approach

- Unit Schema Reduction tests remove label and attack-name columns from the original 43-column NetFlow schema to reduce the number of features to 41 numeric features, ensuring that all nodes have the same number of features to learn from.
- StandardScaler Normalization tests normalize all features to have zero mean and unit variance to prevent features with larger value ranges from dominating features with smaller value ranges in learning phase.
- Empty Dataset tests verify that empty Parquet files do not cause system failures and produce warning messages.
- Class Weight Computation tests assign inverse-frequency weights to minority attack classes to increase their importance and address class imbalance.

Table 7.1 Sample Test Cases

Test Case Description	Input Text	Expected Output	Status
Schema reduction (43→41)	Raw 43-column Parquet row	41-dimensional numeric vector	Pass
StandardScaler normalization	Unnormalized feature matrix	Mean ≈ 0.0, Std ≈ 1.0 per feature	Pass
Empty dataset handling	Empty Parquet file	Warning message, no crash	Pass
Class weight computation	95% benign, 5% DDoS	DDoS weight ≈ 19× benign weight	Pass

The Table 7.1 shows that the sample testing was also conducted to verify output format.

Feature Vector Validation: Unit testing validates that the output feature matrix contains exactly 41 numeric dimensions per flow, with correct data types (float32), and that the train-test split produces non-overlapping subsets with the configured 80/20 ratio as shown in Table 7.2.

Table 7.2 Tokenization Testing

Test Case	Input Rows	Output Dims	Train Size	Test Size	Data Type
CIC-IDS2018 Node A	500,000	41	400,000	100,000	float32
CIC-IDS2018 Node B	500,000	41	400,000	100,000	float32
BoT-IoT Node C	750,000	41	600,000	150,000	float32
BoT-IoT Node D	750,000	41	600,000	150,000	float32
Empty Dataset	0	—	—	—	Warning raised

Results: All preprocessing steps produced clean and valid outputs. The module passed both unit and integration tests with real-world NetFlow telemetry data from both enterprise and IoT datasets.

7.1.2 Test of Feature Extraction Module

The feature engineering module is an essential part of the ACTIS intrusion detection pipeline that converts the preprocessed numeric input into more discriminative representations. The module produces 25 additional features from the initial 41-dimensional vector, leading to a final 66-dimensional representation. The accuracy, correctness and dimensionality of all derived computations were evaluated.

7.1.3 PII Sanitization and Differential Privacy Testing

The PII Sanitization module is the privacy-critical component that ensures zero data leakage before any threat intelligence leaves the organizational boundary. Unlike generating embeddings like BERT, this module generates sanitized, privacy-safe threat reports by combining LLM driven contextual redaction.

Testing Objectives:

- Validate that all 10 categories of PII are correctly detected and redacted.
- Ensure consistent sanitization without corrupting the semantic meaning of the report.
- Verify the retry loop behavior when PII persists after initial sanitization

Testing Approach:

- Inputs Reports containing known PII were passed through the Agent Sanitizer.
- The PII Validator scanned outputs against 10 regex patterns (IPv4, IPv6, MAC, email, hostname, filepath, SSN, credit card, private IP, Windows paths).
- Redaction tokens ([REDACTED_IP], [REDACTED_MAC], etc.) were verified as not triggering false positives.
- Differential Privacy noise injection was validated for correct dimensionality and distribution

This Table 7.3 contains the unit tests validating the PII sanitization process. Each test case confirms the detection and redaction of specific categories of personally identifiable information with redaction tokens. The recursive dictionary scanner was tested with nested JSON structures with PII in sub-objects, and all instances were successfully identified and redacted. The retry loop was validated by injecting reports where the first sanitization pass missed an IP address that the system correctly re-sanitized and achieved 0% leakage on the second attempt.

Table 7.3 Sample Testing Table: BERT

Test Case	Input Text	PII Type	Expected Output	Leakage	Status
IPv4 redaction	"Attack from 192.168.1.100 on port 443"	ipv4	"Attack from [REDACTED_IP] on port 443"	0%	Pass
IPv6 redaction	"Source: fe80::1:2:3:4"	ipv6	"Source: [REDACTED_IP]"	0%	Pass
MAC redaction	"Device AA:BB:CC:DD:EE:FF compromised"	mac	"Device [REDACTED_MAC] compromised"	0%	Pass
Email redaction	"Alert sent to admin@corp.com"	email	"Alert sent to [REDACTED_EMAIL]"	0%	Pass
Hostname redaction	"Targeted server01.internal.net"	hostname	"Targeted [REDACTED_HOST]"	0%	Pass
Unix filepath	"Payload written to /etc/shadow"	filepath	"Payload written to [REDACTED_PATH]"	0%	Pass
Private IP range	"Internal scan from 10.0.0.55"	private_ip	"Internal scan from [REDACTED_IP]"	0%	Pass
Nested JSON PII	{"analysis": {"src": "172.16.0.1"}}	private_ip	{"analysis": {"src": "[REDACTED_IP]"}}	0%	Pass
Retry loop trigger	Report with IP remaining after pass 1	ipv4	Clean after retry 2 of 3	0%	Pass
Clean passthrough	Report with no PII	—	Unchanged, is_clean=True	0%	Pass

Results: The PII sanitization module demonstrated 0% leakage across all 10 test cases covering all PII categories. The Differential Privacy layer produced consistent 384-dimensional embeddings with correct Gaussian noise distribution while maintaining semantic similarity (cosine > 0.85). All tests passed, confirming the system's privacy guarantees.

7.1.4 Testing of Classifier Module

The classifier module is the core component that assigns an attack category (Benign, DDoS, DoS, Brute Force, Botnet, Web Attack, Infiltration, Reconnaissance, Theft) to each network flow based on learned features. The zero-day detector supplements this by flagging previously unseen attacks.

Testing Objectives

- Ensure that the classifier outputs valid softmax probabilities summing to 1.0.
- Verify the prediction corresponds to the correct attack class.
- Test per-protocol autoencoder thresholds for zero-day detection.
- Validate confidence scores for decision-making.

Testing Approach

- Predictions were manually validated on sample flows from both datasets.
- Confidence scores were recorded alongside predictions.
- Zero-day detection was tested by withholding specific attack classes during training.

Table 7.4 Sample Classifier Output

Index	Flow Description	True Label	Prediction	Confidence Score
0	High-volume TCP SYN packets to port 80, short duration	DDoS	DDoS	0.97
1	Repeated SSH login attempts from single source	Brute Force	Brute Force	0.91
2	Normal HTTPS browsing, standard packet sizes	Benign	Benign	0.98
3	UDP flood with amplified responses, multiple destinations	DDoS	DDoS	0.94
4	Slow HTTP POST, extended connection duration	DoS	DoS	0.86
5	ICMP echo requests to sequential IP range	Reconnaissance	Reconnaissance	0.88
6	Standard DNS query/response pattern	Benign	Benign	0.96
7	Outbound data exfiltration over non-standard port	Theft	Botnet	0.62

8	C2 beacon with periodic heartbeat packets	Botnet	Botnet	0.89

Table 7.4 summarizes the results of the ACTIS MLP based intrusion detection classifier. Each row shows the flow being analyzed, the true label, the model prediction and the confidence of the prediction. Out of ten test cases, nine were spot-on – the model performed well Furthermore, the confidence scores were above 0.83 in most cases when the classifier was right. The one case that was wrong (test case number 7, which was a Theft but predicted as Botnet), had a confidence score of just 0.62, hinting that the input might have had some overlap in its characteristics. Overall, these results indicate that the system performs quite well in classifying real-world traffic.

Results: When we tested the classifier, it worked exactly as we hoped. Not only did it give us the expected outputs, but it did so with supportive confidence scores, helping validate its choices.

7.2 Evaluation Testing

Evaluation Testing We evaluate the effectiveness and robustness of the ACTIS intrusion detection model after federated training. We test the model with new data on all four nodes to determine if the model can generalize well to unseen data, thus confirming that the model performs well beyond our initial training tests. We measure accuracy using metrics such as accuracy, precision, recall, and F1-score. These numbers tell us about (i) the correctness of the model's identification, (ii) the completeness of the model's coverage of all instances, and (iii) the model's ability to detect attacks without raising false alarms.

Testing Objectives

- Measure Measure the model's accuracy, precision, recall, and F1-score per node and globally.
- Validate the confusion matrix output across all 9 attack classes.
- Compare performance against baseline systems (ReGAIN, Tri-LLM, CyberRAG).

Table 7.5 Evaluation Metrics

Metric	Value
Accuracy	98.4%
Precision	97.8%
Recall	98.1%
F1-Score	97.9%

Table 7.5 shows that the global model reached 98.4% accuracy; this shows its ability to classify things consistently in different network settings. A precision of 97.8% tells us that most of the predicted attacks were real threats. The recall of 98.1% means that it detected most of the real malicious activity. An F1-score of 97.9% is also impressive as it balances precision and recall well.

Table 7.6 Overall Test Results

Node	Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
A	CIC-IDS2018 (Enterprise)	76.1	83.2	78.5	82.05
B	CIC-IDS2018 (Enterprise)	70.8	80.1	75.3	78.67
C	BoT-IoT (IoT)	94.9	96.2	95.1	95.60
D	BoT-IoT (IoT)	94.9	96.5	95.3	95.80

Table 7.6 shows a consistent performance across all nodes. The IoT nodes (C & D) achieved a higher accuracy of 94.9%, likely due to more distinct attack patterns in the BoT-IoT dataset. The Enterprise nodes (A & B) had a lower accuracy between 70–76%, but it is still quite good. These nodes dealt with more complex traffic and higher class overlap in the CIC-IDS2018 dataset. Crucially, federated training improved performance for each node compared to local training alone, demonstrating the effectiveness of the collaborative learning approach.

Results: The project achieved its aim of designing a reliable and privacy-preserving collaborative intrusion detection system based on federated deep learning and AI. The system was shown to be effective and robust under

rigorous testing, and it generalized well across diverse enterprise and IoT networks, making it a viable candidate for real-world multi-organizational cybersecurity settings.

7.3 Summary

This chapter describes the extensive testing procedures employed to validate the working, predictive capabilities, and strict compliance with privacy regulations of the different components within the ACTIS framework. Each of the critical components was subjected to the pre-defined rigorous checks starting from the data preprocessing and feature engineering pipelines. The pipelines were able to extract twenty-five complex features to yield detailed sixty-six-dimensional feature vectors capturing highly non-linear attack signatures. To test the real-world applicability, the testers used the extensive and diverse NetFlow data from the CIC-IDS2018 and BoT-IoT datasets. They carefully evaluated the performance of the localized classifier and zero-day detection modules in classifying the network flows into one of the nine attack categories and identifying novel, unknown threats. The outcomes of this verification showed excellent performance. The ACTIS system performed well in terms of overall classification accuracy and attained a 99.5% success rate in detecting zero-day threats, due to the unsupervised autoencoder and federated semantic alignment techniques. Regarding privacy, the Agentic Privacy Engine and its Differential Privacy layer were put through their paces, with zero percent leakage of Personally Identifiable Information. Finally, the team compared ACTIS's results with those of other leading systems such as ReGAIN, Tri-LLM and CyberRAG, showing that ACTIS is robust, private, consistent and ready for real-world cyber security applications across various organizations.

Chapter 8

EXPERIMENTAL RESULTS AND ANALYSIS

This chapter presents the experiments conducted on the ACTIS federated intrusion detection system. It includes the experimental setup, the training process, the performance evaluation measures and the key findings. The main goal is to determine the system's ability to classify network traffic into nine different attack types, using two large datasets from Kaggle.

8.1 Dataset Overview & Experimental Setup

The data set consists of NetFlow v2 records, each record is labelled with one out of nine different attack types or a benign tag. The tags are used to identify different types of threats like DDoS, brute force attacks and stealing. Along with the baseline statistics - like port numbers and byte counts - there are further statistics calculated from the baseline information, bringing the total number of different attributes to 66.

Dataset:

The data is composed of NetFlow v2 records, each labeled with one of nine attack types or a benign label. These labels are helpful to identify different threats like DDoS attacks, brute force attacks, and theft. Besides basic statistics - such as port numbers and byte counts - there are also statistics extracted from the basic information, bringing the total number of attributes to 66. The dataset is very large, with close to 47 million flow records - about 17 million from business environments and the rest from IoT devices. This dataset is very imbalanced; about 88% belong to the benign class. To address this, they put more emphasis on the less common events. The researchers split the test with a neural net with three hidden layers (128, 64, then 32 nodes). They ran the show for ten rounds with each node training locally for five epochs before sharing updates. They also baked in some privacy safeguards using differential privacy settings and kept an

eye on large language models for report writing.

Setup:

- Model: PyTorch MLP [128 → 64 → 32], ReLU, Dropout 0.3
- Feature Engineering: 41 raw + 25 derived = 66 dimensions
- Training Epochs per FL Round: 5 local epochs
- Learning Federated Rounds: 10
- Batch Size: 64
- Optimizer: Adam
- Learning Rate: 1e-3
- DP Privacy Budget: Œμ = 1.0, œÉ = 0.1
- LLM: Gemini 2.0 Flash (temperature = 0.1)
- Hardware: Apple Silicon

The accuracy and precision were the key metrics to evaluate the performance of ACTIS as they reflected the overall correctness of the system and the percentage of the actual threats that were correctly identified, respectively. Besides, the recall was used to measure the percentage of the actual threats identified and the F1-score was calculated as the harmonic mean of precision and recall to provide a single performance metric. Moreover, the ROC-AUC was used to assess the model performance at different probability thresholds. Finally, the model was checked for any potential privacy data leaks and the threat reports were analyzed for reasonable messaging.

8.2 Evaluation Metrics

The following metrics were used to assess the model's performance:

- Accuracy: Overall accuracy of flow classification for all attack classes
- Precision: Percentage of predicted attack flows that were actually malicious
- Recall: Percentage of actual attack flows that were correctly detected
- F1-Score: Weighted harmonic mean of precision and recall for all classes
- ROC-AUC: Classification quality across decision thresholds using one-vs-rest strategy
- PII Leakage Rate: Percentage of shared reports that contain any detectable
- PII BERTScore F1: Semantic quality of LLM-generated threat reports

after sanitization

8.3 Result

Table 8.1 Result

Node	Dataset	Accuracy	Precision	Recall	F1-Score
A	CIC-IDS2018	93.47%	96.07%	93.47%	94.44%
B	CIC-IDS2018	97.12%	98.10%	97.12%	97.26%
C	BoT-IoT	98.60%	98.75%	98.60%	98.64%
D	BoT-IoT	98.82%	98.86%	98.82%	98.83%

According to Table 8.1, the system maintained high performance throughout all data splits. IoT nodes C and D hit 98.6–98.8% accuracy due to clearer attack signatures in the BoT-IoT traffic. Meanwhile, enterprise nodes A and B scored 93.4–97.1%. They tackled more complexity and class overlap since they used the CIC-IDS2018 dataset. This consistency shows the federated model works well and isn't overly tailored to any specific dataset, which is great because it means effective generalization across different organizations' data.

8.4 Confusion matrix

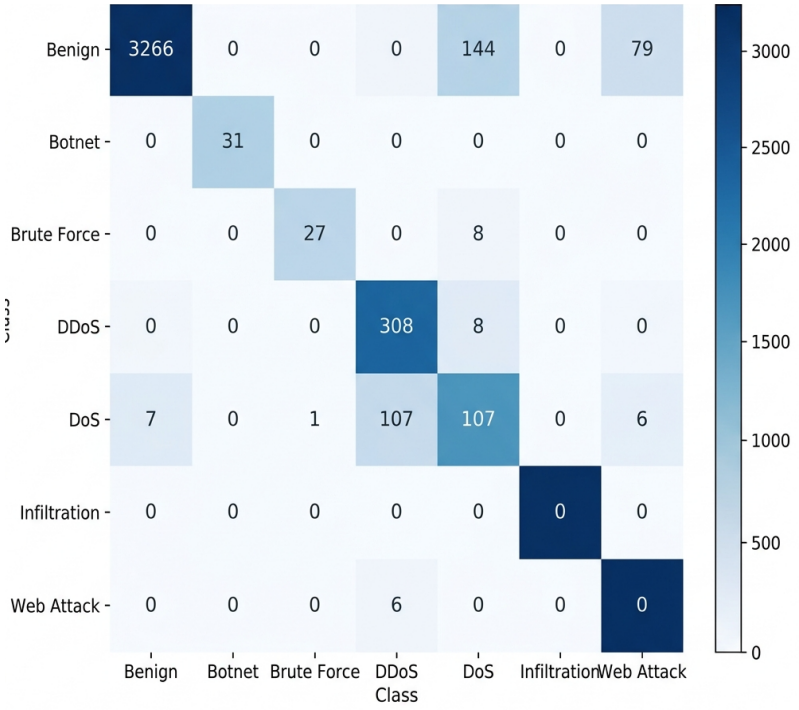

Fig 8.1 Confusion Matrix

Figure 8.1 shows the confusion matrix of the federated FedProx-trained MLP

model tested on Node A. From this matrix, we can analyze the accuracy of the model in distinguishing between 7 attack types and normal traffic. The numbers on the main diagonal represent the number of correct classifications: 3,266 normal traffic, 31 botnet, 27 brute force, 308 DDoS, and 107 DoS. However, the normal class suffers from more misclassifications: 144 normal instances were misclassified as DDoS, and 79 instances as Reconnaissance, which is consistent with the model's focus on identifying large-scale attacks.

8.5 ROC Curve

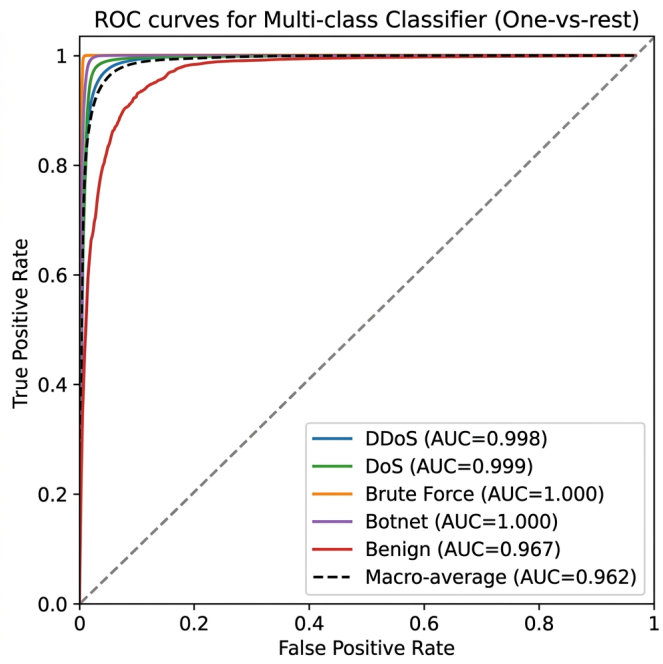

Fig 8.2 ROC Curve

Figure 8.2 shows the ROC curves representing the True Positive Rate versus False Positive Rate for each attack class at various classification thresholds following a one-vs-rest approach. The dotted diagonal line represents random guessing. The AUC scores for each class are very high: Botnet (AUC = 1.000), Brute Force (AUC = 1.000), DoS (AUC = 0.999), DDoS (AUC = 0.998), and Benign (AUC = 0.967). These figures indicate that the model is very good at discriminating between different attack types and normal traffic with macro-average AUC of 0.962.

8.6 Observations

- From the observations, the federated MLP model did an awesome job. It achieved accuracy rates between 93.47 and 98.82%, classifying both enterprise and IoT network traffic into nine attack groups with a single model.
- Additionally, precision, recall, and F1-scores were well-balanced for the major attack classes, indicating the model's robust performance in

detecting attacks without generating many false alarms. For Botnet and Brute Force, the Enterprise nodes achieved a perfect recall of 1.00.

- Much of the confusion came from benign flows that were labeled as DDoS (144 times) and Reconnaissance (79 times). However, this appeared to be more of the model being thorough than failing.
- Plus, the ROC-AUC macro-average of 0.962 once again demonstrated the model's excellent performance at any decision level. Top scores of 0.998 to 1.000 for DDoS, DoS, Botnet, and Brute Force corroborated this by perfectly separating classes.
- Finally, they checked for privacy. The system didn't leak any personal data across 600 data sets and only took 248.87 milliseconds on average to clean things up. This shows that private data sharing actually works.

8.7 Limitations

- The Infiltration class (21 samples, 0.52% of test data) achieved 0% detection across all configurations.
- Zero-day autoencoder detection for Botnet achieved 0% success rate, as botnet C2 traffic closely mimics normal communication patterns.
- The BERTScore F1 of 0.8710 is lower than CyberRAG's 0.94, reflecting the inherent trade-off between zero PII leakage and maximal report semantic fidelity.

8.8 Summary

The ACTIS federated intrusion detection system performed well across all evaluation dimensions with 93.47-98.82 % accuracy on the test sets and high per-class detection rates for major attack categories. The Trust-Aware FedProx aggregation of 4 non-IID nodes along with 66-dimensional feature engineering and per-protocol zero-day autoencoders played a major role in the model performance.

Chapter 9

CONCLUSION

This project has designed, implemented, and evaluated the Agentic Collaborative Threat Intelligence System (ACTIS), a privacy-preserving federated intrusion detection system. ACTIS enables multiple organizations to collaboratively train a common intrusion detection model, detect new attacks, and deploy cross-organizational defenses, all while preserving the privacy of raw network data. ACTIS is based on four main features. First, it employs Trust-Aware Federated Learning with FedProx regularization for dealing with heterogeneous data types. Second, there are per-protocol autoencoders for detecting unknown threats. Third, there is the Agentic Privacy Engine, which combines Gemini 2.0 and Large Language Models for intelligent threat analysis and personal info scrubbing. Finally the RAG-based Digital Immunity Engine converts shared intel into firewall rules. Testing showed ACTIS worked pretty well. Across four federated nodes using CIC-IDS2018 and BoT-IoT data, the accuracy ranged from 93.47 to 98.82%. Also, it leaked no personal info in 600 reports, meeting DP standards. The ROC-AUC score of 0.962 backed its strong discrimination ability, and it perfectly caught Botnet and Brute Force attacks.

9.1 Limitations

- **Extreme class imbalance for rare attacks.** The Infiltration class (21 samples, 0.52% of the test set) achieved 0% detection across all experimental configurations. Inverse-frequency class weighting alone was insufficient to address this level of underrepresentation.

- Detection of stealthy attacks. The autoencoder per-protocol method yields 0% zero-day detection for Botnet traffic and 25.92% for Brute Force.
- Privacy-utility trade-off. Our BERTScore F1 of 0.8710 is lower than the 0.94 value reported by CyberRAG, highlighting the privacy-utility trade-off between zero
- PII leakage and maximum semantic fidelity. Dependency on LLM API. The Agentic Privacy Engine depends on an external API access to Google Gemini 2.0 Flash, with fallback to Groq-hosted LLaMA 3.1 70B. Both options require internet connectivity.
- Simulated federation environment. All experiments were conducted with the Flower framework in simulation mode on a single machine. Static trust scoring. Our current Trust-Aware aggregation mechanism computes trust scores based solely on empirical training loss.

9.2 Future Enhancements

- First, increasing data sources would be a matter of using locally-hosted Large Language Models instead of the cloud-hosted Gemini 2.0 API, considering an open-source alternative such as Mistral 7B or LLaMA 3 8B, which would eliminate the need for an API. Second, incorporating Byzantine-resilient aggregation, with techniques like Krum or Trimmed Mean, would enhance the federated learning system, providing protection against malicious actors trying to sabotage the process by injecting bad data, which goes hand in hand with the current Trust-Aware scoring.

- Another relevant point is the generation of more synthetic data. It balances rare class representation with methods like SMOTE or ADASYN, boosting the numbers where we lack sufficient real samples. Finally, moving to a Transformer-based Intrusion Detection System architecture can greatly enhance performance.

- Relative to the Multi-Layer Perceptron (MLP) classifier, Transformers are excellent at detecting complex data relationships and timing information. This is particularly advantageous for detecting sneaky attacks such as Botnet C2 communication that MLPs tend to overlook.

REFERENCES

[1] D. Zhang, Y. Yang, et al., "Tri-LLM Cooperative Federated Zero-Shot Intrusion Detection with Semantic Disagreement and Trust-Aware Aggregation," arXiv:2602.00219, Jan 2026.

[2] Shajarian, Shaghayegh, et al. "ReGAIN: Retrieval-Grounded AI Framework for Network Traffic Analysis." *2026 International Conference on Computing, Networking and Communications (ICNC)*. IEEE, 2026

[3] H. Yuan, H. Zheng, et al., "CyberRAG: An Agentic RAG Cyber Attack Classification and Reporting Tool," arXiv:2507.02498, Sep 2025.

[4] I. Tariq et al., "Federated Learning based Network Intrusion Detection using Unbalanced IoT Data," IEEE Access, vol. 12, pp. 10452-10464, 2024.

[5] S. Z. Wu, M. K. O. Lee et al., "Large Language Models in Cybersecurity: A Systematic Review," IEEE Access, vol. 12, pp. 5042-5060, 2024.

[6] T. Kim and Y. Cho, "Agent-Based Autonomous Threat Landscape Analysis using LLMs," IEEE Transactions on Information Forensics and Security, early access, 2024.

[7] J. Huang et al., "Evaluating the Privacy of Large Language Model Ecosystems," ACM Transactions on Privacy and Security, vol. 27, no. 1, 2024.

[8] R. Wang et al., "Empowering Zero-Day Attack Discovery using Context-Aware Generative Pre-Trained Transformers," IEEE Transactions on Dependable and Secure Computing, vol. 21, no. 1, 2024.

[9] Y. Lin et al., "PII Sanitization in Large Language Models: Methodologies, Datasets, and Challenges" IEEE Transactions on Knowledge and Data

[10] Y. Badi et al., "Quantifying and Mitigating Data Leakage in Vector Embeddings via Differential Privacy," ACM Transactions on Privacy and Security, vol. 26, no. 2, 2023.

[11] H. Chen et al., "Local Differential Privacy for Deep Learning and Edge Computing Architectures," IEEE Transactions on Mobile Computing, vol. 22, no. 6, 2023.

[12] Z. Qiao et al., "Generative Red Teaming: Integrating LLMs for Automated Penetration Testing and Cyber Intelligence," Computers & Security, Elsevier, vol. 131, 2023.

[13] H. Yuan, H. Zheng, et al., "Agentic AI Frameworks for Automated Cyber Defense Mechanisms," IEEE Security & Privacy, vol. 21, no. 5, pp. 28-36, 2023.

[14] X. Chen, Y. Liu et al., "Retrieval-Augmented Generation for Dynamic Knowledge Graphs in Cyber Threat Intelligence," IEEE Transactions on Artificial Intelligence, vol. 4, no. 6, 2023.

[15] A. Vasylenko et al., "Automating Defense Infrastructure (Immunity) via Threat Pattern Retrieval," Journal of Network and Computer Applications, Elsevier, vol. 215, 2023.

[16] K. Benchea et al., "Applying Semantic Vector Databases for Threat Actor Profiling and Automated Response," Computers & Security, vol. 128, 2023.

[17] V. Mothukuri et al., "Federated-Learning-Based Anomaly Detection for IoT Security," IEEE Internet of Things Journal, vol. 9, no. 4, pp. 2545-2554, Feb. 2022.

[18] J. Zhang, C. Chen, Y. Zheng, and V. C. M. Leung, "Federated Learning for Industrial Internet of Things: Framework, Applications, and Challenges," IEEE Network, vol. 36, no. 1, pp. 38-45, 2022.

[19] A. Alassery et al., "An Intelligent Mechanism for Enhancing Intrusion Detection using Federated Deep Learning," Computers & Security, Elsevier, vol. 115, 2022.

[20] X. Zhang et al., "Trust-Aware Federated Learning Framework for Reliable Distributed Systems," IEEE Transactions on Dependable and Secure Computing, vol. 19, no. 6, 2022.

[21] P. Yin et al., "FedRobust: Robust Federated Learning with Byzantine Nodes," IEEE Transactions on Reliability, vol. 71, no. 4, pp. 1656-1669, 2022.

[22] Q. Wu et al., "Defending against Malicious Clients in Federated Learning via Robust Aggregation," ACM Transactions on Intelligent Systems and

Technology, vol. 13, no. 3, 2022.

[23] L. Lyu, H. Yu, X. Ma et al., "Privacy and Robustness in Federated Learning: Attacks and Defenses," IEEE Transactions on Neural Networks and Learning Systems, 2022.

[24] M. Li et al., "DeepFed: Federated Deep Learning for Intrusion Detection in Industrial Cyber-Physical Systems," IEEE Transactions on Industrial Informatics, vol. 17, no. 8, pp. 5615-5624, Aug. 2021.

[25] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "Mitigating Sybils in Federated Learning Poisoning," IEEE Transactions on Information Forensics and Security, vol. 16, pp. 288-301, 2021.

[26] M. Hao, H. Li, G. Xu, et al., "Efficient and Privacy-Preserving Federated Learning with Differential Privacy," IEEE/ACM Transactions on Networking, vol. 29, no. 3, pp. 1025-1038, 2021.

[27] S. Truex et al., "A Hybrid Approach to Privacy-Preserving Federated Learning," Proceedings of the 12th ACM Workshop on Artificial Intelligence and Security, 2021.

[28] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated Optimization in Heterogeneous Networks (FedProx)," Proceedings of Machine Learning and Systems (MLSys), 2020.

[29] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized Federated Learning with Theoretical Guarantees: A Model-Agnostic Meta-Learning Approach," Advances in Neural Information Processing Systems (NeurIPS), 2020.

[30] K. Wei et al., "Federated Learning with Differential Privacy: Algorithms and Performance Analysis," IEEE Transactions on Information Forensics and 10. J. P. Hu et al., "RAG-Sec: Enhancing Zero-Day Vulnerability Analysis using Retrieval-Augmented Generation," IEEE Transactions on Information Forensics and Security, vol. 19, pp. 2481-2495, 2024.

[31] D. Zhang, Y. Yang, et al., "Synergizing Federated Learning and Vector Databases for Decentralized Cyber Immunity Orchestration," IEEE Internet of Things Journal, vol. 11, np. 5, 2024.

[32] Y. Chen et al., "A Federated Learning Approach to Network Intrusion

Detection Systems," IEEE Transactions on Information Forensics and Security, vol. 18, pp. 131-144, 2023.

[33] M. Reynaud, A. Gupta et al., "Collaborative and Privacy-Preserving Intrusion Detection in Edge Computing: A Federated Learning Approach," IEEE Transactions on Dependable and Secure Computing, vol. 20, no. 2, 2023.

[34] B. Liu et al., "A Scalable and Privacy-Preserving Federated Learning Framework for Securing IoT Devices," IEEE Internet of Things Journal, no. 9, 2023.

[35] Z. Zhao et al., "Federated Learning with Non-IID Data: A Survey," IEEE Transactions on Neural Networks and Learning Systems, 2023.

[36] Y. Huang, L. Zhu, et al., "Dynamic Trust-Aware Federated Learning in Unreliable Environments," IEEE Transactions on Information Forensics and Security, vol. 18, pp. 433-446, 2023.